

Advanced Algorithm Design (AAD)

Prof. G. Pucci

	Mondays	14:30–15:15	15:20–16:05	Pe
Classes	Tuesdays	10:30–11:15	11:20–12:05	Pe
	Wednesdays	12:30–13:15	13:20–14:05	Pe

5 min BREAK

Office Hours (DEI/G, room 414) Wednesdays: 15:30–17:00. Compulsory reservation by (same) wednesday morning via mail: geppino.pucci@unipd.it.

MOODLE: registration is compulsory! Follow regularly for announcements, slides, etc.

Password: —

WEBPAGE: www.dei.unipd.it/~geppo/AAD. Surf to get to know the sections.

Advanced Algorithm Design (AAD)

First Semester, 2026/27. Laurea Magistrale in Computer Engineering.

- Instructor ← Info about me, office hours.
- Syllabus ← High level description of course topics.
- Textbooks.
- Lectures Diary A.Y. 2026/27: Lecture-by-lecture — This year.
- Lectures Diary A.Y. 2025/26: Lecture-by-lecture — Last year.
- Exam Rules.
- Exam Sessions ← Dates/rooms already fixed!
- Online Material ← Supplementary material.

INFO ALSO ACCESSIBLE FROM MOODLE.

Objective: Integrate and expand the knowledge acquired in an introductory course on algorithms. The material is mainly theoretical but has enormous practical impact.

Topics

- NP-Completeness: more concrete approach (non determinism — verifiability).
– REDUCTION TECHNIQUES.
- Approximation algorithms for hard problems (solutions of guaranteed quality).
- Computational aspects of number theory (divisibility, modular arithmetic, primality) and cryptographic applications of hardness.
- Randomization: general techniques for designing randomized algorithms.

IMPORTANT: Emphasis on rigorous proof techniques, general paradigms but will ALWAYS discuss the practical relevance of the material.

(Main) Textbook

- CLRS (Introduction to Algorithms 3rd ed): Cormen, Leiserson, Rivest, Stein.
- Abundant supplementary material (see web page).

Attendance Bonus +1 pt for those students attending ≥ 30 lectures.

EXAM: written, two parts

Part 1: theory questions on covered material (proofs, definitions, analyses): 30%.

Part 2: creative exercises apply techniques to new problems (lots of exercise sessions during lectures): 70%.

Voluntary TEAMWORK: groups of 2 students. Read and understand advanced material, prepare a short written report plus class-presentation (with slides).

EVALUATION: Exemption from Part 1 and 30% of final grade.

IMPORTANT: Embark into teamwork only if **STRONGLY** motivated.

→ team creation + assignment: first weeks of december.

→ presentation must be done in January (last week of lectures), Jan 11–13.

PREREQUISITES (Discrete Math & Algorithms)

- Asymptotic notation: O, Ω, Θ .
- Summations, basic probability (distributions, averages, variance, random variables).
- Divide-and-conquer: design (recursion), analysis (recurrences — MASTER THEOREM).
- Elements of Dynamic Programming and Greedy.

If these topics are not in your background you should acquire them by yourself (use CLRS: Chaps 3, 4, 15, 16, App A, B, C).

I will be glad to help students with difficulties (use office hours/email).

END (course intro)

COMPLEXITY CLASSES AND THEORY OF INTRACTABILITY (NP-COMPLETENESS)

Objective: CATEGORIZE computational problems according to the complexity of the algorithms for their solution.

RECAP: Computational Problem

$$\Pi \subseteq \mathcal{I} \times \mathcal{S} \quad (\text{relation}), \quad (i, s) \in \Pi \quad [i \Pi s] \quad \Longleftrightarrow \quad s \text{ is a solution to } i.$$

Many-to-many!

Algorithm that solves Π : implements

$$A_\Pi : \mathcal{I} \longrightarrow \mathcal{S}, \quad \forall i \in \mathcal{I} : \quad i \Pi A_\Pi(i) \in \mathcal{S}.$$

We are interested in upper/lower bounds (on algorithms) for Π :

$$\underbrace{\exists A_\Pi \text{ solving } \Pi : T_A(n) = O(f(n))}_{\text{UPPER BOUND}},$$
$$\underbrace{\forall A_\Pi \text{ solving } \Pi : T_A(n) = \Omega(f(n))}_{\text{LOWER BOUND (HARDER!)}}.$$

The order of $f(n)$ captures the difficulty of Π .

PROBLEM CATEGORIES

1. Problems admitting polynomial algorithms: TRACTABLE.

$$\exists A_{\Pi} : T_{A_{\Pi}}(n) = O(n^k), \quad k \geq 0.$$

2. Problems admitting an exponential lower bound: PROVABLY INTRACTABLE.

$$\exists c > 1 : \forall A_{\Pi} : T_{A_{\Pi}}(n) = \Omega(c^n).$$

3. Problems not admitting any algorithm (e.g. HALTING PROBLEM): UNDECIDABLE/UNSOLVABLE.

$$\Pi_{\text{HALT}} : \mathcal{I} = \{\langle M, x \rangle : M \text{ Turing Machine}, x \in \{0, 1\}^*\}, \quad \mathcal{S} = \{\text{yes}, \text{no}\}.$$

$$\langle M, x \rangle \Pi_{\text{HALT}} \text{ yes} \iff M(x) \text{ halts.}$$

4. Intractable problems: neither an exponential l.b. or a polynomial u.b. are known!

- Problems of huge practical interest (economics, electronics, AI, learning, biology, logistics).
- Strong structural properties: a polynomial algorithm for any single problem yields polynomial algorithms for all!

NP-Completeness theory.

We'll deal with classes of problems: we need to standardize computational problems as much as possible!

ABSTRACT DECISION PROBLEMS

$$\Pi_D \subseteq \mathcal{I} \times \mathcal{S}, \quad \mathcal{S} = \{\text{yes}, \text{no}\}.$$

Formulation of an abstract decision problem:

- description of the generic instance;
- question on the instance.

Yes/no answer is the solution.

EXAMPLE: Membership of integer key in a set

SEARCH:

$$\begin{cases} \text{INSTANCE: } \langle S, m \rangle : S \subseteq \mathbb{N}, \text{ finite}, m \in \mathbb{N}, \\ \text{QUESTION: } m \in S? \end{cases}$$

OBS: A decision problem is a function

$$\Pi_D : \mathcal{I} \longrightarrow \{\text{si}, \text{no}\}.$$

Positive/negative instances.

Restriction to decision problems: excessive?

In practice we usually solve more complex optimization problems!

OPTIMIZATION PROBLEM

$$\Pi_{\text{opt}} \subseteq \mathcal{I} \times \mathcal{S}, \quad c : \mathcal{S} \longrightarrow \mathbb{N} \cup \{+\infty\}.$$

$\forall i \in \mathcal{I}$: determine $s^* \in \mathcal{S}(i) = \{s : i \Pi s\}$ (FEASIBLE SOLUTIONS):

$$c(s^*) = \min/\max\{c(s) : s \in \mathcal{S}(i)\}.$$

EXAMPLE: SHORTEST-UNWEIGHTED PATH (SUP)

$$\mathcal{I} = \{\langle G = (V, E), u, z \rangle : V \subseteq \mathbb{N}, \text{ finite}, E \subseteq V \times V, u, z \in V\}.$$

$\mathcal{S} = \mathbb{N}^*$ (sequences of naturals: paths).

- $i \Pi s$ if

$$(s = \langle v_1, v_2, \dots, v_k \rangle, v_i \in V) \wedge (v_1 = u, v_k = z) \wedge (v_i, v_{i+1}) \in E, \quad 1 \leq i < k,$$

where $i = \langle G = (V, E), u, z \rangle$.

- $i \Pi \varepsilon$ if $\nexists$ path from u to z in G .

$$c(\langle v_1, \dots, v_k \rangle) = k - 1, \quad k \geq 1; \quad c(\varepsilon) = +\infty.$$

Minimization problem.

An optimization problem can be easily associated to a decision problem using the following standard technique: augment instance with a natural value representing an upper (MIN) or lower (MAX) limitation to the objective function, for all feasible solutions.

EXAMPLE: k -UNWEIGHTED-PATH (KUP)

$$\begin{aligned} \text{I: } & \langle G = (V, E), u, z, k \rangle : V \subseteq \mathbb{N}, \text{ finite}, E \subseteq V \times V, \\ & u, z \in V, k \in \mathbb{N}. \end{aligned}$$

$$\text{D: } \exists \text{ path from } u \text{ to } z \text{ of length } \leq k?$$

NOTE: The decision problem is not fully equivalent but it holds that:

A polynomial algorithm for KUP can be used as a subroutine to solve SUP in polynomial time!

Let $A_{\text{KUP}}(G = (V, E), u, z, k)$ be a polynomial time algorithm for KUP.

We first show that A_{KUP} can be employed to determine the length of the shortest path between u and z as follows:

P-LENGTH($\langle G = (V, E), u, z \rangle$)

```

 $n \leftarrow |V|$ 
for  $k \leftarrow 0$  to  $n - 1$  do
  if  $A_{\text{KUP}}(\langle G = (V, E), u, z, k \rangle) = \text{yes}$ 
    then return  $k$ 
return  $+\infty$ 

```

Clearly, P-LENGTH(G, u, z) returns the smallest value of k such that $\exists$ a path from u to z ($+\infty$ if no path exists).

Since $|V| = n \leq |\langle G, u, z \rangle|$ (SIZE OF THE INSTANCE) and

$$|\langle G, u, z, k \rangle| = \Theta(|\langle G, u, z \rangle|),$$

if $T_{\text{KUP}}(\langle G, u, z, k \rangle) = O(|\langle G, u, z, k \rangle|^c)$ then

$$T_{\text{P-LENGTH}}(\langle G, u, z \rangle) = O(|\langle G, u, z \rangle|^{c+1}).$$

Once the length k^* of the shortest path is established, it is easy to determine the actual path through $|E|$ calls to A_{KUP} .

Algorithm idea: progressively eliminate edges which are not necessary to guarantee the existence of a shortest path. To check this, we use A_{KUP} as an ORACLE.

$A_{\text{SUP}}(\langle G = (V, E), u, z \rangle)$

```

 $k^* \leftarrow \text{P-LENGTH}(\langle G, u, z \rangle)$  {see above}
if ( $k^* = +\infty$ ) then return "no path"
*  $E = \{e_1, e_2, \dots, e_{|E|}\}$  *
 $E' \leftarrow E$ 
for  $i \leftarrow 1$  to  $|E|$  do
  if  $A_{\text{KUP}}(\langle G' = (V, E' - \{e_i\}), u, z, k^* \rangle) = \text{yes}$ 
    then  $E' \leftarrow E' - \{e_i\}$ 
    { $\exists$  shortest path  $\pi^* \in E'$  not using  $e_i$ }

```

* E' contains only the edges of a s.p.!! *

return π^*

Correctness: if $k^* = +\infty$, A_{SUP} is correct. Otherwise, by contradiction, if after the for loop, E' does not contain only the edges of a single shortest path π^* :

$$\exists e_{\bar{i}} \in E' \text{ in } E' \text{ not used by } \pi^*.$$

BUT at iteration $\bar{i}$ we would have:

$$A_{\text{KUP}}(\langle G' = (V, E' - \{e_{\bar{i}}\}), u, z, k^* \rangle) = \text{yes!}$$

Thus $e = e_{\bar{i}}$ would have been removed!

→ We have obtained a poly-time algorithm A_{SUP} for SUP!

(EXERCISE: determine complexity)

Morale: restriction to decision problems is without loss of generality (w.l.o.g.) if we are only interested in determining existence of poly-time algorithms.

RECAP

- Problem classes (categorization with respect to complexity of solution algorithms) → standardization.
- Abstract decision problems: $\Pi_A : \mathcal{I} \rightarrow \{\text{yes}, \text{no}\}$.
- Optimization problems can be paired with a computationally equivalent decision problem → add upper/lower bound on cost to instance.
- If decision problem admits a poly-time algorithm then same holds for optimization problem (AND VICEVERSA).

CAVEAT: We can construct optimization problems where the equivalence does not hold! But it holds for totality of interesting problems.

Morale: Concentrate on decision problems.

Classes of decision problems

- Problems become elements of sets (classes): need a standard format.
- $\mathcal{S}$ is already standardized:

$$\mathcal{S} = \{\text{yes, no}\}, \quad \Pi_A : \mathcal{I} \longrightarrow \{\text{yes, no}\}.$$

- Need to standardize $\mathcal{I}$:
 - From set of abstract structures of arbitrary nature.
 - To set of binary strings $\subseteq \{0, 1\}^*$.

DEF: An encoding is an injective function, mapping an abstract set $\mathcal{I}$ to $\{0, 1\}^*$:

$$e : \mathcal{I} \longrightarrow \{0, 1\}^*.$$

An encoding turns an abstract decision problem Π_A into a concrete decision problem:

$$\begin{array}{ccc} \Pi_A : \mathcal{I} & \longrightarrow & \{0, 1\} \\ \downarrow e(\cdot) & & \\ \Pi_C : \{0, 1\}^* & \longrightarrow & \{0, 1\}. \end{array}$$

The two problems are related as follows:

$$\forall x \in \{0, 1\}^* : \quad (1) \quad \Pi_C(x) = 1 \iff \exists i \in \mathcal{I} : (x = e(i)) \wedge (\Pi_A(i) = \text{yes}).$$

NOTE 1: $\Pi_C(x)$ is defined on all strings in $\{0, 1\}^*$. From $(1, \Rightarrow)$ we have:

$$\forall x \in \{0, 1\}^* : \quad \nexists i \in \mathcal{I} : x = e(i) \Rightarrow \Pi_C(x) = 0.$$

NOTE 2: The size of a concrete instance is $|x|!$

An algorithm A_{Π_C} for a concrete problem takes a binary string as input and outputs 0 or 1.

WARNING: The choice of the encoding is a delicate issue with critical impact on the complexity analysis of the algorithm.

RECALL: Worst case complexity:

$$T(n) = \max\{\text{cost of instructions executed by } A_{\Pi}(i), \text{ with } \text{size}(i) = n\}.$$

(Standard cost model: each instruction costs 1.)

Given an algorithm A_{Π_C} for a concrete problem Π_C :

$$T_{A_{\Pi_C}}(n) = \max_{|x|=n} \{\# \text{ instructions executed by } A_{\Pi_C}(x)\}.$$

Since $|x| = |e(i)| = n$ (i : abstract instance), the independent parameter is the encoding length!

→ “Unreasonably” lengthy encodings may “mask” the inefficiency of an algorithm!

EXAMPLE: Let $\mathcal{I} = \mathbb{N} - \{0\}$.

BINARY ENCODING: If $m \in \mathcal{I} = \mathbb{N} - \{0\}$,

$$e_b(m) = \langle a_k, a_{k-1}, \dots, a_0 \rangle, \quad a_k = 1,$$

with $k = \lfloor \log_2 m \rfloor$ and

$$m = \sum_{j=0}^k a_j 2^j.$$

NOTE 1:

$$|e_b(m)| = \lfloor \log_2 m \rfloor + 1 = \Theta(\log m).$$

UNARY ENCODING: If $m \in \mathcal{I} = \mathbb{N} - \{0\}$,

$$e_u(m) = \underbrace{\langle 1, \dots, 1 \rangle}_{m \text{ times}} \quad (\text{string of } m \text{ 1's}).$$

NOTE 2: $|e_u(m)| = m$.

NOTE 3:

$$m = |e_u(m)| = 2^{\log_2 m} = \Theta(2^{|e_b(m)|}).$$

The unary encoding is exponentially longer than the binary encoding!

RIPERCUSSIONS ON COMPLEXITY ANALYSIS

RECALL: $T(n)$ uses encoding length as the independent variable n .

EXAMPLE: Consider this algorithm:

GAUSS(m)

```
sum ← 0
for  $i \leftarrow 1$  to  $m$  do sum ← sum +  $i$ 
return sum
```

GAUSS computes $\sum_{i=0}^m i = m(m+1)/2$ executing m iterations. But

$$m = |e_u(m)| = \Theta(2^{|e_b(m)|}).$$

$\Rightarrow$

$$T_{\text{GAUSS}}^U(\underbrace{|e_u(m)|}_n) = \underbrace{|e_u(m)|}_n \quad (\text{independent parameter}).$$

GAUSS takes linear time in the size of the input under the UNARY encoding!

BUT

$$T_{\text{GAUSS}}^B(\underbrace{|e_b(m)|}_{n'}) = m = \Theta(\underbrace{2^{|e_b(m)|}}_{2^{n'}}).$$

GAUSS takes exponential time in the size of the input under the BINARY encoding!

The same algorithm is associated to drastically different complexities by the two encodings!

QUESTION: Is GAUSS good or bad?

BAD! For $m = 2^{32}$ GAUSS executes $m \simeq 4$ billion sums!

while

GAUSS2(m)

```
return  $m \cdot (m + 1)/2$ 
```

executes 3 operations between numbers of size ≤ 64 .

MORALE: The linear complexity under the unary encoding is MISLEADING. The fact that GAUSS is bad is masked by the sheer length of the unary encoding.

We will use only concise encodings whose length is comparable (i.e., at most polynomially larger) with a minimum-length encoding.

EXAMPLES

- Naturals must be encoded to the base $b \geq 2$, $b = O(1)$.
- A set of naturals is encoded by concatenating the encodings of its elements.

From now on we will ignore the details of the encoding and assume that a concise encoding is in use!

NOTATION: Given a set of abstract instances $\mathcal{I}$: $\forall i \in \mathcal{I}$, $\langle i \rangle \in \{0, 1\}^*$ denotes a concise encoding of i . e.g., $\langle G = (V, E) \rangle$.

Standardization is now complete!

$$\Pi_C : \{0, 1\}^* \longrightarrow \{0, 1\}.$$

ANY concrete decision problem maps binary strings (instance encodings) into bits (1/0 encoding yes/no).

FORMAL LANGUAGE REPRESENTATION OF CONCRETE DECISION PROBLEMS

RECAP: Given an alphabet Σ , a formal language L over Σ is a set of finite strings:

$$L \subseteq \Sigma^* \quad (\text{Kleene star of } \Sigma).$$

- L may/may not contain the empty string ε ($\varepsilon \notin L$ or $\varepsilon \in L$).
- Empty language: $\emptyset$ ($\{\varepsilon\} \neq \emptyset$!).
- $L_1 \cdot L_2$ concatenation operator:

$$L_1 \cdot L_2 = \{z \in \Sigma^* : \exists x \in L_1, y \in L_2 : z = \langle x, y \rangle\}.$$

- Powers: $L^0 = \{\varepsilon\}$; $L^i = L \cdot L^{i-1}$.
- Kleene star of a

$$L^* = \bigcup_{i=0}^{\infty} L^i = \{z \in \Sigma^* : \exists k \geq 1 : \exists x_1, \dots, x_k \in L : z = \langle x_1, x_2, \dots, x_k \rangle\} \cup \{\varepsilon\}.$$

ALL FINITE CONCATENATIONS OF STRINGS $\in L$.

There is a one-to-one correspondence between concrete problems and formal languages over $\Sigma = \{0, 1\}$:

$$\Pi_C \longleftrightarrow L_{\Pi_C} = \{x \in \{0, 1\}^* : \Pi_C(x) = 1\} \quad (\text{encodings of positive instances}).$$

We will use the two concepts interchangeably.

$$\begin{aligned} \text{Problem } \Pi_C &\longleftrightarrow \text{Algorithm solving } \Pi_C, \\ \text{Language } L_{\Pi_C} &\longleftrightarrow \text{Algorithm deciding } L_{\Pi_C} \text{ (Automaton)}. \end{aligned}$$

DEF: A accepts x if $A(x) = 1$. A rejects x if $A(x) = 0$.

DEF:

$$L_A = \{x \in \{0, 1\}^* : A(x) = 1\} \quad (\text{language decided by } A).$$

DEF: A language L is decided in polynomial time if $\exists A$:

1. $L = L_A$.

2. $\exists k \in \mathbb{N} : \forall x \in \{0,1\}^* : T_A(|x|) = O(|x|^k)$ (A runs in polynomial time).

DEF: A complexity class is a set of languages (equivalently: a set of concrete (decision) problems).

DEF:

$$P = \{L \subseteq \{0,1\}^* : L \text{ is decided in polynomial time}\}.$$

ALTERNATIVE DEF: (equivalent)

$$P = \{\Pi_C : \{0,1\}^* \rightarrow \{0,1\} : \Pi_C \text{ is solved in polynomial time}\}.$$

EXAMPLE

SEARCH:

INSTANCE: $\langle S, m \rangle : S \subseteq \mathbb{N} \text{ finite, } m \in \mathbb{N}$,

QUESTION: $m \in S?$

$$L_{\text{SEARCH}} = \{x \in \{0,1\}^* : (x = \langle S, m \rangle) \wedge m \in S\}.$$

$A_{\text{SEARCH}}(x)$

```

if  $(x \neq \langle S, m \rangle)$  then return 0
 $k \leftarrow |S|$ 
*  $S = \{s_1, \dots, s_k\}$  *
for  $i \leftarrow 1$  to  $k$  do
    if  $(s_i = m)$  then return 1
return 0

```

$$T_{A_{\text{SEARCH}}}(|x|) = O(|x|) \Rightarrow \{\text{SEARCH}, L_{\text{SEARCH}}\} \in P.$$

POLYNOMIAL-TIME VERIFICATION

SOLVING

build a solution from scratch

VERIFYING

check that a given candidate is a solution

Example: Recurrences

solving: build the closed formula.

verifying: given a candidate closed formula check its correctness.

Formal languages

solving: algorithm that decides if $x \in L$.

verifying: two-input algorithm: x plus “additional info” y . Algorithm checks that y certifies that $x \in L$ (e.g., y = closed formula).

Verifying is easier than solving!

EXAMPLE

DEF: Given $G = (V, E)$ undirected, a simple cycle is a sequence:

$$\langle v_1, v_2, \dots, v_k \rangle, \quad v_i \in V, \quad v_1 = v_k,$$

$$v_i \neq v_j \quad 1 \leq i < j < k, \quad k \geq 1, \quad \{v_i, v_{i+1}\} \in E \quad 1 \leq i < k.$$

HAMILTON:

I: $\langle G = (V, E) \rangle$, G undirected,

D: is there in G a simple cycle containing all nodes in V ?

Example:



Deciding whether a graph contains a Hamiltonian cycle is a difficult problem (no polynomial algorithms are known).

Verifying that a candidate vertex sequence is indeed a Hamiltonian cycle is easy!

VERIFY-HAMILTON(x, y)

```

if ( $x \neq \langle G = (V, E) \rangle$ ) then return 0
if ( $y \neq \langle v_1, \dots, v_{|V|+1} \rangle$ ,  $v_i \in V$ ) then return 0
{check that input is "well formed"}
if ( $v_1 \neq v_{|V|+1}$ ) then return 0
{candidate cannot be a cycle!}
if ( $\exists i, j : 1 \leq i < j \leq |V| : v_i = v_j$ ) then return 0
{check that  $y$  contains all nodes}
for  $i \leftarrow 1$  to  $|V|$  do
    if ( $\{v_i, v_{i+1}\} \notin E$ ) then return 0
{check cycle edges}
return 1

```

The algorithm is at most quadratic in the size of the input!

DEF: A two-input algorithm V verifies string $x \in \{0, 1\}^*$ iff $\exists y \in \{0, 1\}^* : V(x, y) = 1$. Any $y : V(x, y) = 1$ is called a certificate for x .

For instance, if $x = \langle G = (V, E) \rangle \in L_{\text{HAMILTON}}$, a certificate for x is

$$y = \langle v_1, \dots, v_{|V|+1} \rangle, \quad v_1 = v_{|V|+1}, \\ v_i \neq v_j \quad 1 \leq i < j \leq |V|, \quad (v_i, v_{i+1}) \in E.$$

IMPORTANT: if $x \notin L$ then no certificate may exist!

DEF: The language verified by a two-input algorithm V is:

$$L_V = \{x \in \{0, 1\}^* : V \text{ verifies } x\} \\ = \{x \in \{0, 1\}^* : \exists y \in \{0, 1\}^* : V(x, y) = 1\}.$$

OBSERVATION: VERIFY-HAMILTON is a verification algorithm for L_{HAMILTON} (or, equivalently, for HAMILTON).

IMPORTANT: We only require that one certificate exists but do not ask the algorithm to compute it itself.

The certificate is provided to the algorithm externally. “Computing the certificate” would imply solving the problem.

DEF: A language L is verified in polynomial time iff $\exists V(x, y)$ such that:

1. $L = L_V$.
2. $\exists k_1 \geq 0 : \forall x \in L \exists y : V(x, y) = 1$ and $|y| = O(|x|^{k_1})$.
3. $\exists k_2 \geq 0 : T_V(|x|, |y|) = O((|x| + |y|)^{k_2})$.

We need SHORT CERTIFICATES!

DEF:

$$\text{NP} = \{L \subseteq \{0, 1\}^* : L \text{ verified in polynomial time}\}.$$

NP can be equivalently defined with respect to decision problems.

We proved that HAMILTON $\in$ NP.

Since verifying is easier than solving, we suspect that

$$\text{if } L \in \text{P} \Rightarrow L \in \text{NP}.$$

That is, $\text{P} \subseteq \text{NP}$.

RECAP

- Encodings: $\Pi_A \xrightarrow{\mathcal{I} \rightarrow e(\mathcal{I})} \Pi_C : \{0, 1\}^* \rightarrow \{0, 1\}$.
– reasonable/concise encodings.
- $\mathcal{R}$: Formal languages.

$$\underbrace{\Pi_C}_{\text{solution}} \longleftrightarrow \underbrace{L_{\Pi_C} = \{x \in \{0, 1\}^* : \Pi_C(x) = 1\}}_{\text{decision}}.$$

- Poly-time solution/decision: P.
- Verification: algorithm $V(x, y)$.

$$L_V = \{x \in \{0, 1\}^* : \exists y \in \{0, 1\}^* : V(x, y) = 1\}.$$

- Poly-time verification: NP.
– poly-size certificate $[|y| = O(|x|^{k_1})]$;
– poly-time algorithm $[T_V(x, y) = O((|x| + |y|)^{k_2})]$.

Theorem: $\text{P} \subseteq \text{NP}$ ($L \in \text{P} \Rightarrow L \in \text{NP}$).

Proof: Let $L \in \text{P}$. We’ll prove $L \in \text{NP}$ by constructing a poly-time verifier: Recall that $L \in \text{P} \Rightarrow \exists A_L$ which decides L in time $T_{A_L}(|x|) = O(|x|^k)$. Verifier $V_L(x, y)$ uses A_L as a subroutine:

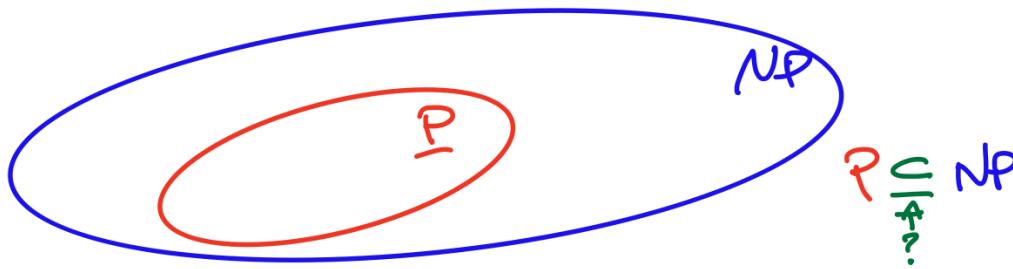
$V_L(x, y)$

return $A_L(x)$

The verifier disregards y and calls $A_L(x)$. Clearly we have

1. $L_V = L$ ($V_L(x, y) = 1 \iff x \in L$).
2. $y = \varepsilon$ is a certificate $\forall x \in L$ ($|\varepsilon| = O(|x|^0) : k_1 = 0$).

3. $T_V(|x|, |y|) = T_{A_L}(|x|) = O(|x|^k) = O((|x| + |y|)^k)$ ($k_2 = k$).



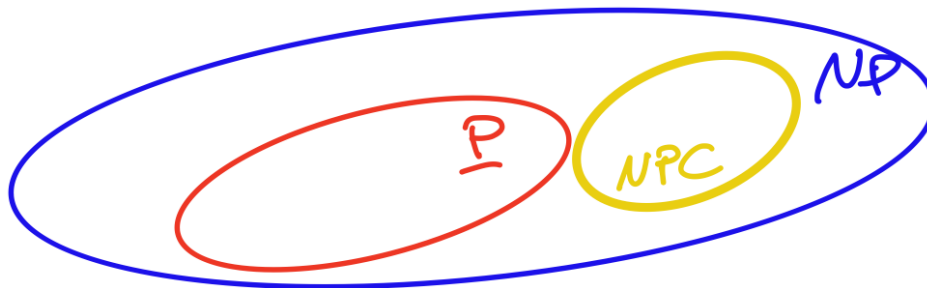
What can we say about $NP - P$? Could be empty! ($P = NP$) Could be nonempty! ($P \neq NP$)

The question $P = NP$? is still open (1 million-dollar question).

Reasonable guess: $P \neq NP$.

Evidence:

1. Verifying seems intrinsically easier than solving.
2. We'll define a partial order in NP (based on solution difficulty). For the most difficult problems in NP no poly-time algorithms are known.
3. The most difficult problems in NP form an "equivalence class": NPC (NP -complete problems).



If $P \neq NP \Rightarrow NPC \subset NP - P$ (LIKELY!)

If $P = NP \Rightarrow NPC \subset P$ (UNLIKELY!)

We will see that:

- NPC contains many decision problems from different fields of huge practical interest.
- Problems in NPC are "equivalent" in the following sense:

A poly-time solution to any one problem in NPC immediately yields a poly-time solution to all problems in NPC !

REDUCIBILITY BETWEEN PROBLEMS

Intuitively, Π_1 can be reduced to Π_2 if any instance i_1 of Π_1 can be "easily" transformed into an instance i_2 of Π_2 so that a solution to i_2 provides a solution to i_1 .

For decision problems:

$$i_1 \mapsto f(i_1) = i_2, \quad \Pi_1(i_1) = \Pi_2(i_2).$$

Map f must transform positive instances into positive instances and negative instances into negative instances.

SIMPLE EXAMPLE

$$\Pi_1 : \begin{cases} \text{I: } \langle a, b, x \rangle, & a, b, x \in \mathbb{C}, \\ \text{Q: } ax + b = 0? \end{cases}$$

(decision version of first degree equations)

$$\Pi_2 : \begin{cases} \text{I: } \langle c, e, f, x \rangle, & c, e, f, x \in \mathbb{C}, \\ \text{Q: } cx^2 + ex + f = 0? \end{cases}$$

(decision version of second degree equations)

We can reduce Π_1 to Π_2 as follows

$$i_1 = \langle a, b, x \rangle \xrightarrow{f} \langle \underbrace{0}_c, \underbrace{a}_e, \underbrace{b}_f, x \rangle = i_2.$$

Clearly, $\Pi_1(i_1) = \Pi_2(f(i_1)) = \Pi_2(i_2)$.

$$0 \cdot x^2 + ax + b = ax + b = 0?$$

NOTE: An “efficient” reduction from Π_1 to Π_2 implies that Π_1 is “less difficult” than Π_2 .

(If I can solve Π_2 efficiently, then I can also solve Π_1 efficiently.)

“Efficient” reducibility creates an order between problems.

DEF: A function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is polynomial-time computable (ptc) if $\exists$ algorithm $A_f : \{0, 1\}^* \rightarrow \{0, 1\}^*$:

$$\forall x \in \{0, 1\}^* : A_f(x) = f(x) \text{ and } T_{A_f}(|x|) = O(|x|^k)$$

for some constant $k \geq 0$.

DEF: L_1 is poly-time reducible to L_2 (notation: $L_1 \leq_p L_2$) if:

$$\exists f : \{0, 1\}^* \rightarrow \{0, 1\}^* \text{ ptc} : \quad x \in L_1 \iff f(x) \in L_2.$$

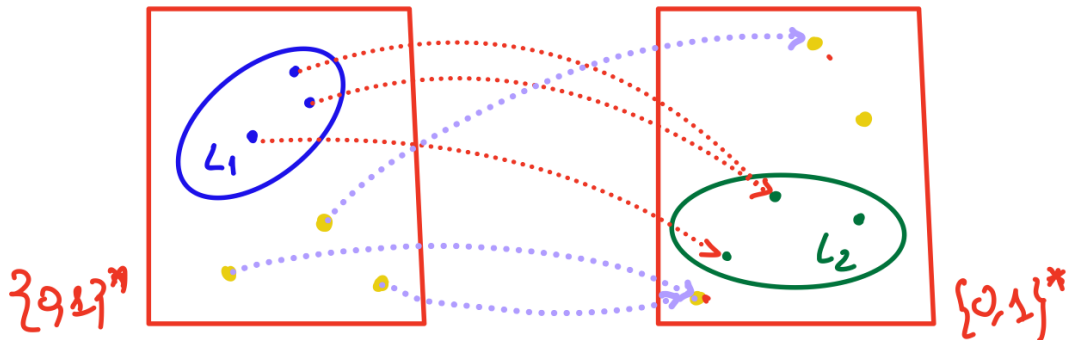
(w.r.t. problems: $\Pi_{L_1}(x) = \Pi_{L_2}(f(x))$)

$\iff$ means that:

- strings in L_1 are mapped to strings in L_2 ($\Rightarrow$);
- strings not in L_1 are mapped to strings not in L_2 ($\Leftarrow \equiv \nrightarrow$: COUNTERPOSITIVE).

WE NEED TO PROVE TWO IMPLICATIONS.

Remark: f need not be neither injective or surjective.



Poly-time reducibility captures the concept of “lesser difficulty” of L_1 with respect to L_2 in the following sense: $L_1 \leq_p L_2$.

THEOREM: Given $L_1, L_2 \subseteq \{0, 1\}^*$:

$$(L_1 \leq_p L_2) \wedge (L_2 \in P) \Rightarrow L_1 \in P.$$

(If L_2 can be decided in poly-time, so can L_1 !)

PROOF: We have two hypotheses:

HP1: $L_1 \leq_p L_2$: $\exists f$ ptc: $x \in L_1 \iff f(x) \in L_2$. Let $T_{A_f}(|x|) = O(|x|^{k_1})$, $k_1 \geq 0$.

HP2: $L_2 \in P$: $\exists A_{L_2}$ which decides L_2 in poly time. Let $T_{A_{L_2}}(|x|) = O(|x|^{k_2})$.

I can write the following algorithm:

$A_{L_1}(x)$

```

     $y \leftarrow A_f(x)$ 
    return  $A_{L_2}(y)$ 

```

Running time:

$$T_{A_{L_1}}(|x|) = O(|x|^{k_1} + |y|^{k_2}).$$

How large can y be? A_f may use all of its steps to create bits of y , thus

$$|y| = O(|x|^{k_1}).$$

Hence

$$T_{A_{L_1}} = O(|x|^{k_1} + |x|^{k_1 k_2}) = O(|x|^{k_3})$$

with $k_3 = \max\{k_1, k_1 \cdot k_2\}$: STILL POLYNOMIAL!

It remains to show that $L_1 = L_{A_{L_1}}$ ($\subseteq$ and $\supseteq$):

$$\begin{aligned} x \in L_1 &\Rightarrow y = f(x) \in L_2 \Rightarrow A_{L_1}(x) = A_{L_2}(y) = 1 \\ &\Rightarrow x \in L_{A_{L_1}} \quad L_1 \subseteq L_{A_{L_1}}. \end{aligned}$$

$$\begin{aligned} x \in L_{A_{L_1}} &\Rightarrow A_{L_1}(x) = A_{L_2}(y) = A_{L_2}(f(x)) = 1 \\ &\Rightarrow f(x) \in L_2 \Rightarrow x \in L_1 \quad L_{A_{L_1}} \subseteq L_1. \end{aligned}$$

We proved $L_1 \subseteq L_{A_{L_1}}$ and $L_1 \supseteq L_{A_{L_1}} \Rightarrow L_1 = L_{A_{L_1}}$.

A_{L_1} is a poly-time algorithm deciding L_1 , therefore $L_1 \in P$.

REMARK: The proof uses both implications $x \in L_1 \iff f(x) \in L_2$.

Poly-time reducibility $\leq_p$ is a relation between languages.

Homework! Prove that:

$$\underbrace{L \leq_p L}_{\text{reflexivity}}, \quad \underbrace{(L_1 \leq_p L_2) \wedge (L_2 \leq_p L_3) \Rightarrow (L_1 \leq_p L_3)}_{\text{transitivity}}.$$

DEF: $L \subseteq \{0, 1\}^*$ is NP-hard if

$$\forall L' \in \text{NP} : L' \leq_p L.$$

$$\text{NPH} = \{L \in \{0, 1\}^* : L \text{ NP-hard}\}.$$

An NP-hard language/problem L is “more difficult” than any problem in NP, in the sense that a poly-time algorithm for L can be transformed into a poly-time algorithm for any problem in NP.

REMARK: An NP-hard language L need not be in NP itself!

DEF: $L \subseteq \{0, 1\}^*$ is NP-Complete if

1. $L \in \text{NP}$.
2. $L \in \text{NPH}$.

$$\text{NPC} = \{L \subseteq \{0, 1\}^* : L \text{ NP-complete}\}.$$

REMARK: Clearly, $\text{NPC} \subseteq \text{NP}$.

NP-complete problems are the “hardest” problems in NP! In fact, they are all somewhat equivalent, since

$$\forall L_1, L_2 \in \text{NPC} : \left(\underbrace{L_1}_{\in \text{NP}} \leq_p \underbrace{L_2}_{\in \text{NPC}} \right) \wedge \left(\underbrace{L_2}_{\in \text{NP}} \leq_p \underbrace{L_1}_{\in \text{NPC}} \right).$$

COROLLARY:

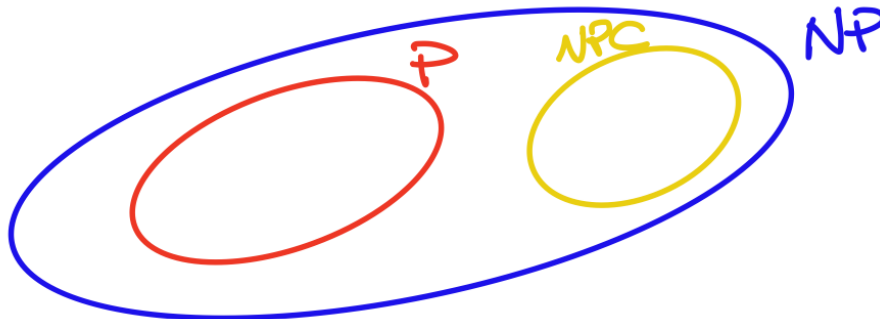
$$\text{If } \exists \bar{L} \in (\text{NPC} \cap \text{P}) \Rightarrow \text{P} = \text{NP}.$$

PROOF: $\forall L \in \text{NP}$:

$$(L \leq_p \bar{L}) \wedge (\bar{L} \in \text{P}) \xrightarrow{\text{Theorem}} L \in \text{P}.$$

Thus, $\text{NP} \subseteq \text{P}$, but also $\text{P} \subseteq \text{NP} \Rightarrow \text{P} = \text{NP}$.

No poly-time algorithms are known for NP-complete problems. Evidence points towards $\text{P} \cap \text{NPC} = \emptyset$. If this is the case:



We will often prove results under the hypothesis that $\text{P} \neq \text{NP}$ (or, equivalently, $\text{P} \cap \text{NPC} = \emptyset$).

NP-COMPLETENESS PROOFS

We have defined NPC but how do we place problems in NPC?

Given a candidate language $\bar{L}$:

1. Prove $\bar{L} \in \text{NP}$.
2. Prove that $\forall L \in \text{NP} : L \leq_p \bar{L}$.

1. is usually easy (come up with a verification algorithm) (e.g., HAMILTON).
2. more difficult because of $\forall$! Infinite reductions!

Fortunately, $\leq_p$ is transitive!

If I know that a given language $\hat{L} \in \text{NPC}$, it suffices to prove that

$$\underbrace{\hat{L}}_{\text{Known to be in NPC}} \leq_p \underbrace{\bar{L}}_{\text{Candidate to be in NPC}}.$$

If $\hat{L} \leq_p \bar{L}$, since $\forall L \in \text{NP} : L \leq_p \hat{L}$ we have:

$$\forall L \in \text{NP} : (L \leq_p \hat{L}) \wedge (\hat{L} \leq_p \bar{L}) \xrightarrow{\text{transitivity of } \leq_p} \forall L \in \text{NP} : L \leq_p \bar{L}.$$

A single reduction suffices!

Unfortunately, the transitivity trick cannot be used for the first language/problem to be placed in NPC! The first problem requires a “universal” reduction from a generic problem in NP!

RECAP

- Classes P and NP. $P \subseteq \text{NP}$.
- Poly-time reducibility: $L_1 \leq_p L_2$. “relative difficulty” of problems:

$$\underbrace{(L_1 \leq_p L_2)}_{\text{“LESS DIFFICULT”}} \wedge (L_2 \in P) \Rightarrow L_1 \in P.$$

- Classes NPH and NPC:

$$L' \in \text{NPC} : (L' \in \text{NP}) \wedge \underbrace{(\forall L \in \text{NP} : L \leq_p L')}_{\text{NPH}}.$$

- Facts:

$$(P \cap \text{NPC} \neq \emptyset) \Rightarrow P = \text{NP}.$$

$$L_1, L_2 \in \text{NPC} : (L_1 \leq_p L_2) \wedge (L_2 \leq_p L_1).$$

- Membership in NPC: use transitivity of $\leq_p$ by proving:

$$L_{\text{KNOWN-NPC}} \leq_p L_{\text{CANDIDATE}}, \quad \forall L \in \text{NP}, L \leq_p L_{\text{KNOWN-NP}}.$$

- First language in NPC requires direct application of definition: ∞ reductions!

In 1971 Cook proved the first NP-completeness result on a problem in boolean logic. We’ll work with an equivalent variant defined on boolean circuits.

DEF: A gate is an elementary switching circuit of three types:

1. AND



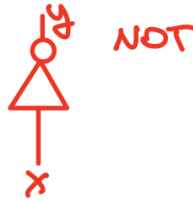
output wire $y = 1 \iff$ input wires $x_1, x_2, \dots, x_k$ all carry 1. Thus $y = \bigwedge_{i=1}^k x_i$.

2. OR



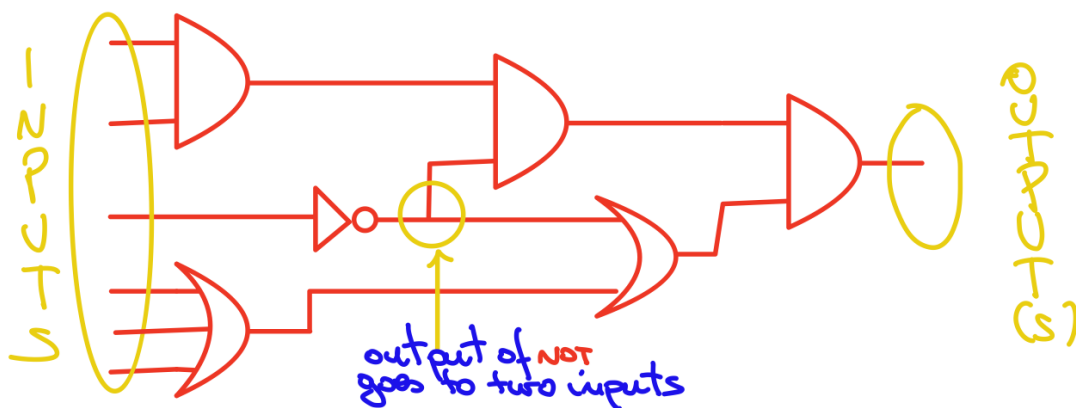
output wire $y = 1 \iff$ at least one input wire carries 1. Thus $y = \bigvee_{i=1}^k x_i$.

3. NOT



output wire $y = 1 \iff$ input wire $x = 0$. Thus $y = \neg(x)$ ($= \bar{x}$).

DEF: A Boolean Circuit (BC) is an acyclic interconnection of gates, where the output of a gate can become input to one or more other gates.



Wires that do not originate from the output of a gate are called inputs of the circuit.

Wires that do not enter into a gate are called outputs of the circuit. We will mostly consider circuits with a single output.

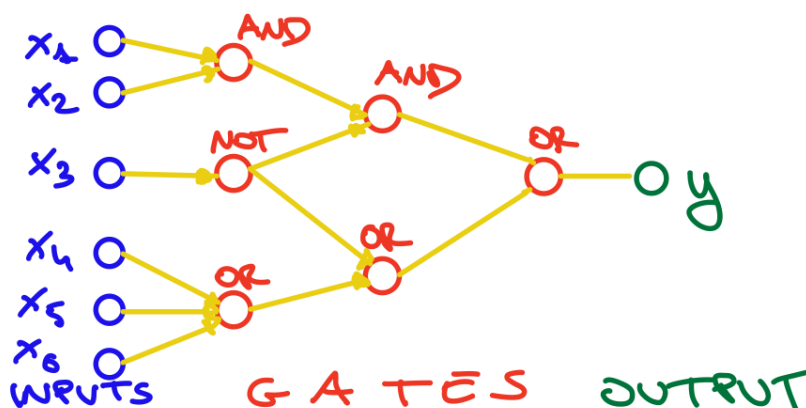
FAN-IN: max # wires entering a gate.

FAN-OUT: max # gates fed by the same wire.

A circuit can be represented by a directed graph

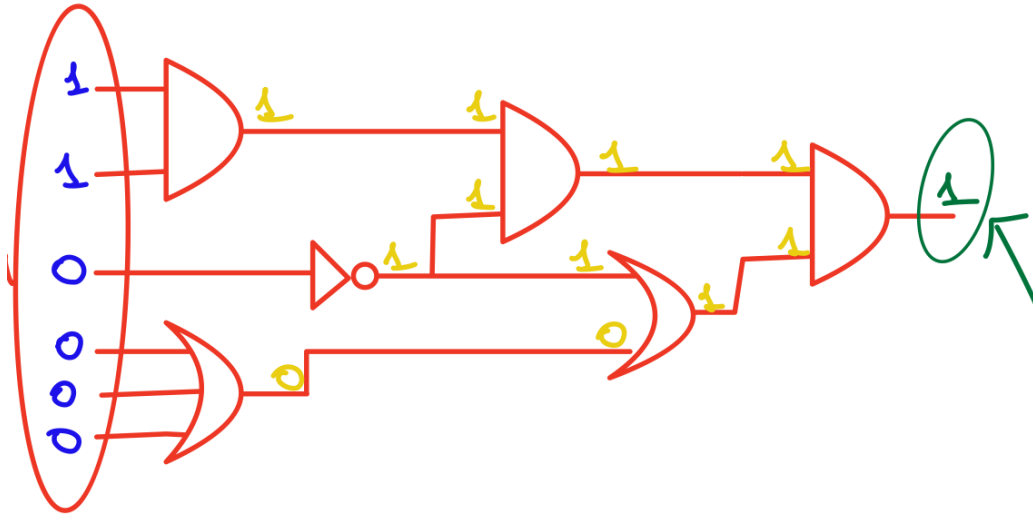
$$C = (I \cup G \cup O, W).$$

- One labeled node per input, gate, output.
- One edge for each wire.



DEF: A truth-assignment to the inputs of a circuit associates a boolean value to each input.

Under a truth-assignment each wire in the circuit will switch to the correct boolean value according to the gates traversed:

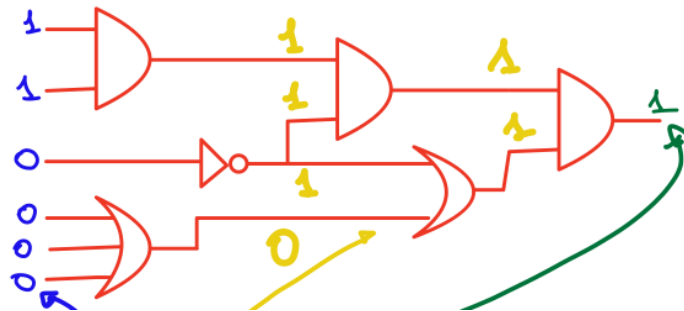


The circuit implements a boolean function $y = C(x_1, x_2, \dots, x_n)$.

For a circuit with n inputs, a truth assignment is $\vec{b} \in \{0, 1\}^n$.

DEF: The set of boolean values carried by all wires of a circuit C under a truth-assignment $\vec{b}$ is called Configuration γ_C of C under $\vec{b}$.

DEF: A boolean circuit $C(x_1, x_2, \dots, x_n)$ is satisfiable if there exists a truth assignment $\vec{b} \in \{0, 1\}^n : C(\vec{b}) = 1$.



C is satisfied by the truth assignment $\vec{b} = (1, 1, 0, 0, 0, 0)$.

Configuration: $(1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1) = \gamma_C$ (in some prespecified order).

BOOLEAN CIRCUIT SATISFIABILITY (BC-SAT)

I: $\langle C(x_1, \dots, x_n) \rangle = \langle (I \cup G \cup O, W) \rangle$, boolean circuit,

Q: C is satisfiable? $\exists \vec{b} \in \{0, 1\}^n : C(\vec{b}) = 1$?

BC-SAT is an important problem in practice (e.g.: testing, circuit equivalence ...). It is easy to prove that

THEOREM: BC-SAT $\in$ NP.

Proof:

VERIFY-BCSAT(x, y) {high-level}

if $x \neq \langle I \cup G \cup \{y_0\}, W \rangle$ then return 0

if $y \neq \langle w_1, w_2, \dots, w_{|W|} \rangle \in \{0, 1\}^{|W|}$

then return 0

{“candidate” certificate looks like a configuration γ_C (1 value/wire)}

if $\exists w_i \neq w_j$ outgoing from same gate

then return 0 {incoherent configuration!}



for each gate $g \in G$

if (I/O relation for g incorrect) then return 0

e.g.:



{check that the configuration is coherent}

return wire value associated to y_0

{ $= 1 \Rightarrow y = \gamma$ certifies that C is satisfiable}

$$T_{VB}(|x|, |y|) = \Theta(|x| + |y|) \Rightarrow \text{BC-SAT} \in \text{NP}.$$

Difficult part:

THEOREM (Cook, 1971)

BC-SAT $\in$ NPH.

The proof of Cook's theorem is complex in the details but relies on a simple intuition.

INFORMAL PROOF

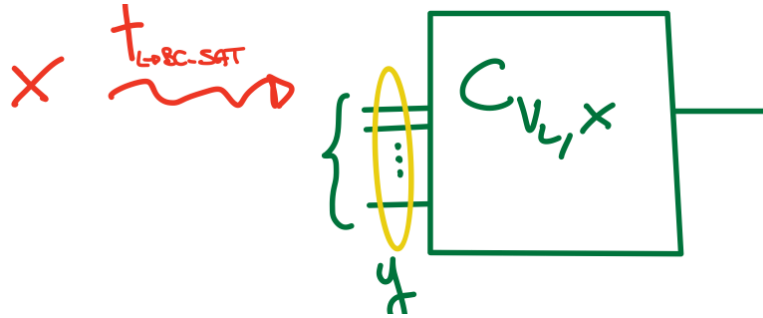
We need to show that given an arbitrary $L \in \text{NP}$, $L \leq_p L_{\text{BC-SAT}}$.

Given L , the only thing that we know is that $\exists V_L(x, y)$, poly-time verifier for L .

We must define a reduction function

$$f_{L \rightarrow \text{BC-SAT}}(x) : \{0, 1\}^* \longrightarrow \{0, 1\}^*.$$

The reduction function builds a boolean circuit starting from V_L : as follows



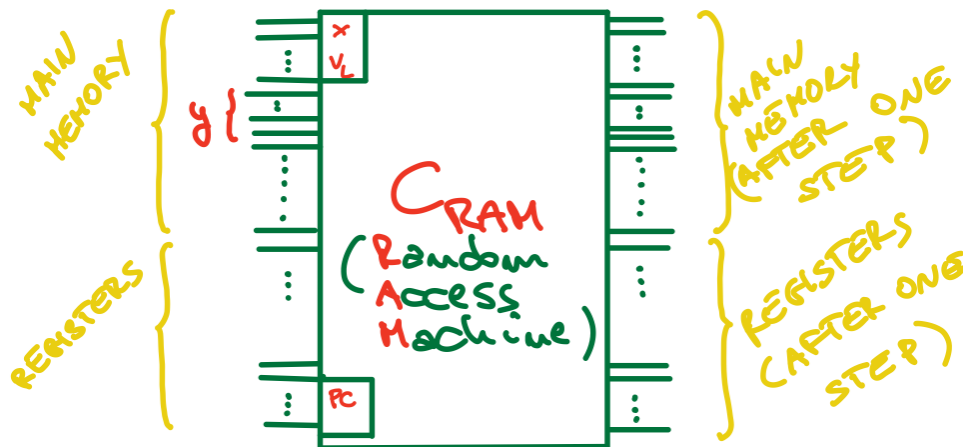
The shape of $C_{V_L, x}$ depends on V_L and x . Its inputs “represent” input y of V_L .

We will make sure that:

$$\begin{aligned} f(x) = \langle C_{V_L, x} \rangle \in L_{\text{BC-SAT}} \text{ (is satisfiable)} &\iff \exists \text{ certificate } y \text{ of poly-length for } x \\ &\iff x \in L. \end{aligned}$$

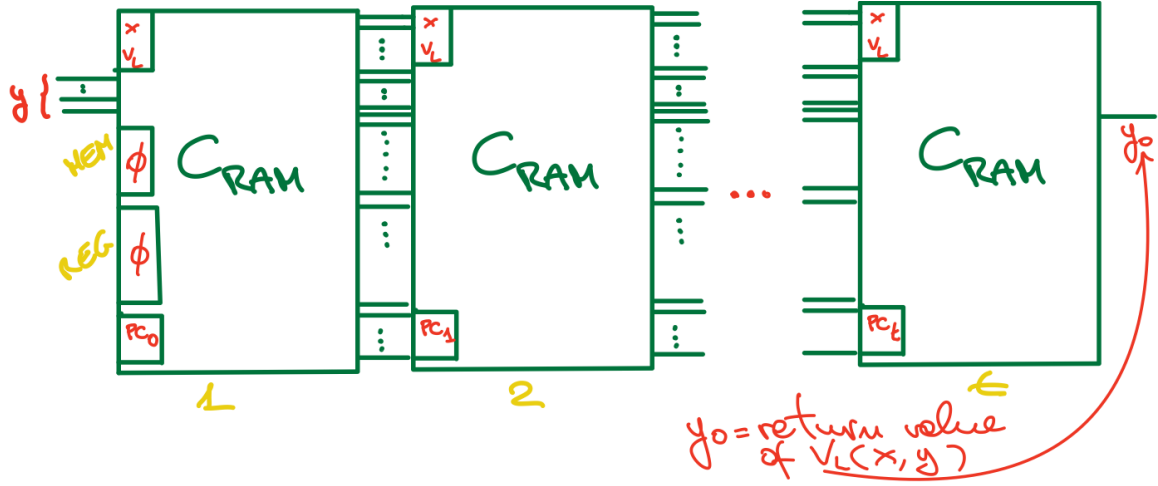
IDEA: $C_{V_L, x}(y)$ “simulates” the computation performed by $V_L(x, y)$.

$C_{V_L, x}$ is built starting from a combinatorial circuit executing a single step of a Random Access Machine (RAM).



C_{RAM} implements the standard circuitry of a RAM (ALU, branches, etc. ...). Memory is “simulated” by passing along stored values on output wires.

An algorithm executing t steps can be simulated by cascading t copies of C_{RAM} :



Crucial points:

1. $|\langle C_{RAM} \rangle|$ is polynomial in x and y .
 - circuitry polynomial in the memory size.
2. It suffices to set $|y| = \Theta(|x|^{k_1})$.
3. It suffices to consider $t = \Theta((|x| + |y|)^{k_2})$.

(k_1, k_2 from definition of poly-time verifier.)

$\Rightarrow C_{V_L, x}$ has size polynomial in x ! Therefore $f_{L \rightarrow BC-SAT}$ is PTC.

We have

$$\begin{aligned}
 x \in L &\iff \exists \bar{y}, |\bar{y}| = O(|x|^{k_1}) : V_L(x, \bar{y}) = 1 \\
 &\iff C_{V_L, x}(\bar{y}) = 1 \\
 &\quad \{C_{V_L, x} \text{ simulates } V_L(x, y)\} \\
 &\iff \langle C_{V_L, x} \rangle \in BC-SAT \\
 &\iff f_{L \rightarrow BC-SAT}(x) \in BC-SAT.
 \end{aligned}$$

Thus $L \leq_p BC-SAT$. This proves that $BC-SAT \in NPH$.

In what follows we will prove that a number of problems are in NPC. The details of the reductions will teach us general approaches and highlight pitfalls to avoid.

FORMULA SATISFIABILITY (SAT)

A boolean formula $\phi(x_1, x_2, \dots, x_n) : \{0, 1\}^n \rightarrow \{0, 1\}$ is a (parenthesized) expression built on

- n variables $x_1, x_2, \dots, x_n$;
- binary operators: $\vee$ (OR), $\wedge$ (AND), $\rightarrow$ (IMPLIES), $\leftrightarrow$ (EQUALS);
- unary operator: $\neg$ (NOT).

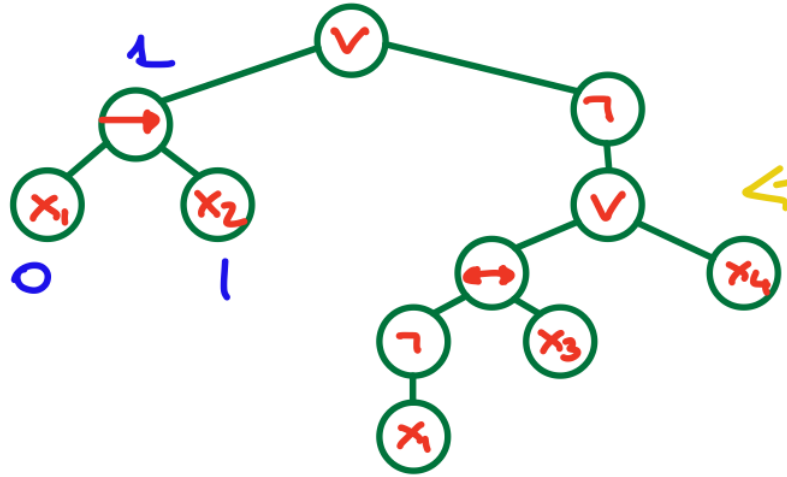
x, y	$x \rightarrow y$	$x \leftrightarrow y$
00	1	1
01	1	0
10	0	0
11	1	1

$$x \rightarrow y \equiv (\neg x) \vee y, \quad x \leftrightarrow y \equiv (x \wedge y) \vee (\neg x \wedge \neg y).$$

EXAMPLE:

$$\phi(x_1, x_2, x_3, x_4) = (x_1 \rightarrow x_2) \vee (\neg((\neg x_1 \leftrightarrow x_3) \vee x_4)).$$

Can be represented by the parenthesization tree (leafs: variables; internal nodes: operators).



A truth-assignment is a vector of boolean values $\vec{b} \in \{0, 1\}^n$, one for each variable.

The value $\phi(\vec{b})$ can be computed in time linear in the formula's size by using the tree.

A satisfying assignment $\vec{b}$ is such that $\phi(\vec{b}) = 1$. If $\vec{b}$ exists, ϕ is said satisfiable.

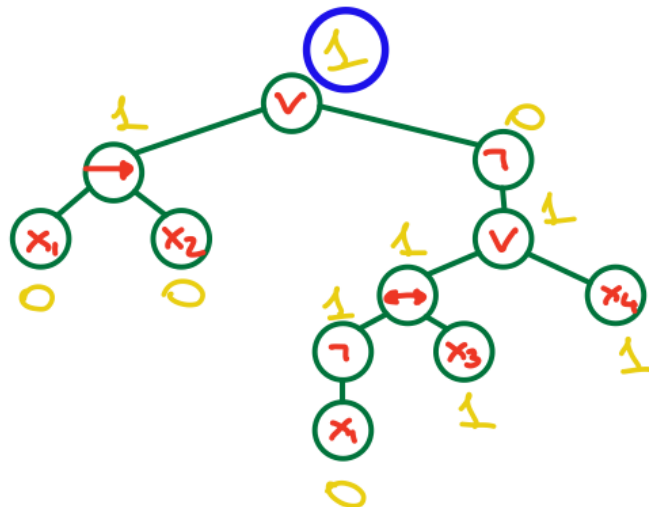
FORMULA SATISFIABILITY (SAT)

I: $\langle \phi(x_1, \dots, x_n) \rangle$, ϕ boolean formula,

Q: is ϕ satisfiable?

EXAMPLE: $\vec{b} = (0, 0, 1, 1)$.

$$\phi(x_1, x_2, x_3, x_4) = (x_1 \rightarrow x_2) \vee (\neg((\neg x_1 \leftrightarrow x_3) \vee x_4)).$$



ϕ is satisfiable [$\phi(\vec{b}) = 1$].

Very similar to BC-SAT (in fact, this is Cook's original problem). Nonetheless, the reduction is nontrivial.

1. SAT $\in$ NP: Trivial: for $\langle \phi(x_1, \dots, x_n) \rangle$ a candidate certificate is $\vec{b} \in \{0, 1\}^n$. $\phi(\vec{b})$ can be computed in linear time using the tree representation.

(write code of VERIFY-SAT(x, y) as an exercise)

2. SAT $\in$ NPH. We show that BC-SAT $\leq_p$ SAT.

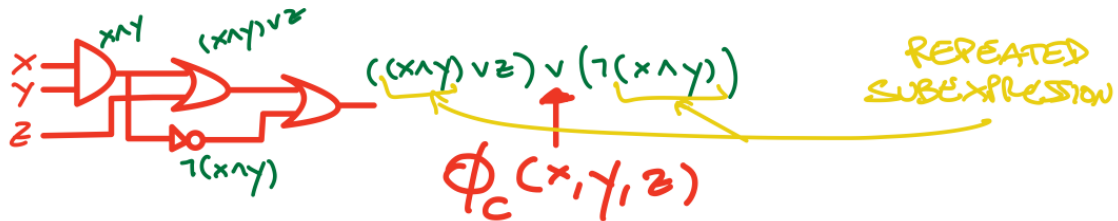
Reduction function f : must transform a circuit C into a formula ϕ_C :

$$C \text{ is satisfiable} \iff f(C) = \phi_C \text{ is satisfiable,}$$

$$\langle C \rangle \in \text{BC-SAT} \iff \langle \phi_C \rangle = f(\langle C \rangle) \in \text{SAT.}$$

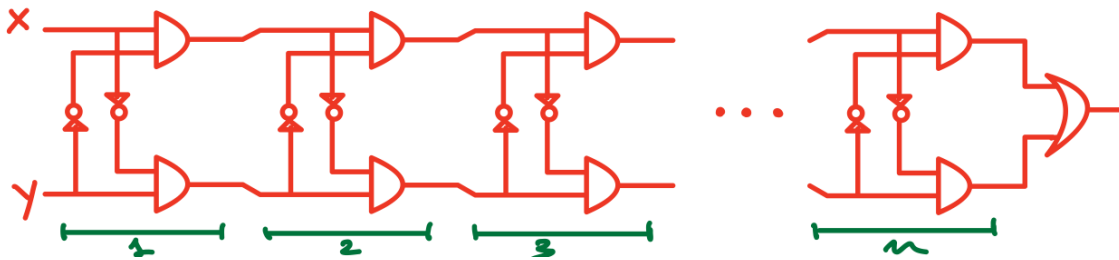
Seems easy: straightforward method: compute the subexpression associated to each wire. ϕ_C = expression associated to output wire.

EXAMPLE:



PROBLEM: Repeated subexpressions may blow-up the length of the formula exponentially!

CONSIDER:



Two-input circuit made of n stages. $|C(x_1, x_2)| = \Theta(n)$.

Outputs of Stage 1: $(x \wedge (\neg y))$, $((\neg x) \wedge y)$.

Outputs of Stage 2:

$$(x \wedge (\neg y)) \wedge (\neg((\neg x) \wedge y)), \quad (\neg(x \wedge (\neg y))) \wedge ((\neg x) \wedge y),$$

etc.

The number of symbols in the subexpressions of Stage i is more than double than that of Stage $i - 1$.

$$\Rightarrow \begin{cases} \text{Size}(i) \geq 2 \text{Size}(i - 1), \\ \text{Size}(1) = c. \end{cases}$$

$$\Rightarrow |\phi_C(x, y)| \geq \text{Size}(n) \geq c \cdot 2^{n-1}!$$

OBSERVE! $|C| = \Theta(n)$ but $|\phi_C(x, y)| = \Omega(2^n)$.

This reduction cannot be ptc! (I can't generate an exponential number of symbols in polynomial time!)

We need a more sophisticated approach!

$$C = (\underbrace{I}_{x_i} \cup \underbrace{G}_{y_g \forall g \in G} \cup \underbrace{\{y_0\}}_{y_0}, W).$$

New variables $y_g : g \in G$ (associated to G).

RECAP

- COOK'S THEOREM: BCSAT $\in$ NPH.
 - “simulate” $V_L(x, y)$ with $C_{V_L, x}(y)$.
 - A certificate y_x for $x \in L$ becomes a satisfying assignment for $f(x) : \langle C_{V_L, x}(y) \rangle$.
- FORMULA SATISFIABILITY (SAT): BC-SAT $\leq_p$ SAT.
 - Trivial reduction based on subexpressions not ptc (poly-time computable).

We need a more sophisticated approach!

$$C = (\underbrace{I}_{x_i} \cup \underbrace{G}_{y_g \forall g \in G} \cup \underbrace{\{o\}}_{y_0}, W).$$

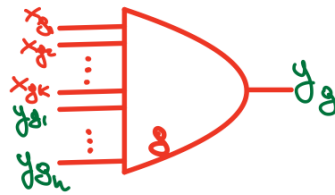
New variables $y_g : g \in G$.

We add extra variables y_g associated to the gates, describing the output of each gate $g \in G$ and create a formula that reflects valid configurations of the circuit.

RECALL: A configuration is the set of all bits on the wires under a valid switching of the circuit: $(\underbrace{\vec{b}_1}_{\vec{x}}, \underbrace{\vec{b}_2}_{\vec{y} \text{ (gate outputs)}})$.

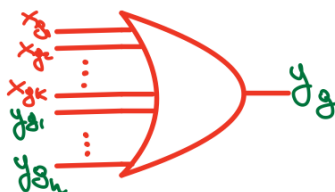
For each gate $g \in G$ we introduce a sub-formula Γ_g describing the input-output relation of the gate:

- AND gate:



$$\Gamma_g : y_g \stackrel{\leftrightarrow}{\text{“equation”}} x_{g1} \wedge \cdots \wedge x_{gk} \wedge y_{g1} \wedge \cdots \wedge y_{gn}.$$

- OR gate:



$$\Gamma_g : y_g \leftrightarrow x_{g_1} \vee \cdots \vee x_{g_k} \vee y_{g_1} \vee \cdots \vee y_{g_h}.$$

- NOT gate:



$$\Gamma_g : y_g \leftrightarrow \neg(z_{g_1}).$$

Observe that subexpression Γ_g is true under a truth assignment iff y_g is assigned the correct output value w.r.t. the values assigned to the gate's inputs.

We can then obtain the formula

$$\psi(\vec{x}, \vec{y}) = \bigwedge_{g \in G} \Gamma_g.$$

Formula ψ describes a configuration of the circuit C :

$$\begin{aligned} \psi(\vec{b}_1, \vec{b}_2) = 1 &\iff \text{the values of the outputs of the gates of } C(\vec{b}_1) \text{ are } (b_2)_g : g \in G \\ &\iff (\vec{b}_1, \vec{b}_2) \text{ is a valid configuration.} \end{aligned}$$

OBSERVATION: $\psi(\vec{x}, \vec{y})$ is always satisfiable (HOMEWORK: how many satisfying assignments?)

We have that

$$|\langle \psi(\vec{x}, \vec{y}) \rangle| = \Theta(|\langle C(\vec{x}) \rangle|)$$

since each subexpression uses a number of symbols linear in the size of the corresponding gate!

The reduction function f uses ψ as follows:

$$f(\langle C(\vec{x}) \rangle) = \phi_C(\vec{x}, \vec{y}) = \psi(\vec{x}, \vec{y}) \wedge \underbrace{y_0}_{\text{variable associated to gate of output wire}}.$$

$$|\langle \phi_C(x, y) \rangle| = \Theta(|\langle \psi(\vec{x}, \vec{y}) \rangle|) = \Theta(|\langle C(x) \rangle|).$$

Thus f is ptc.

We are left to prove that $x \in L_1 \iff f(x) \in L_2$.

$$\left\{ \begin{array}{l} x \in L_1 \Rightarrow f(x) \in L_2, \\ f(x) \in L_2 \Rightarrow x \in L_1 \end{array} \right. \quad \text{OR} \quad \left\{ \begin{array}{l} x \in L_1 \Rightarrow f(x) \in L_2, \\ x \notin L_1 \Rightarrow f(x) \notin L_2. \end{array} \right.$$

$$\langle C(\vec{x}) \rangle \in \text{BC-SAT} \iff f(\langle C(\vec{x}) \rangle) = \langle \phi_C(\vec{x}, \vec{y}) \rangle \in \text{SAT}.$$

($\Rightarrow$) If $\langle C(\vec{x}) \rangle \in \text{BC-SAT}$ then $\exists \vec{b}_1 \in \{0, 1\}^n : C(\vec{b}_1) = 1$. Consider the configuration of the circuit under $\vec{b}_1$, and let $\vec{b}_2$ be the values carried by the wires at the outputs of the gates. We have $\psi(\vec{b}_1, \vec{b}_2) = 1$. Moreover, since $C(\vec{b}_1) = 1$ we have $\underbrace{b_0}_{y_0} = 1$.

$$\Rightarrow \phi_C(\vec{b}_1, \vec{b}_2) = \psi(\vec{b}_1, \vec{b}_2) \wedge b_0 = 1 \Rightarrow f(\langle C(\vec{x}) \rangle) \in \text{SAT}.$$

$$(\Leftarrow) f(\langle C(\vec{x}) \rangle) \in \text{SAT} \Rightarrow \exists \vec{b}_1, \vec{b}_2 : \phi_C(\vec{b}_1, \vec{b}_2) = 1.$$

1. $\psi(\vec{b}_1, \vec{b}_2) = 1$: the truth assignments represent a legal configuration $(\vec{b}_1, \vec{b}_2)$ of the circuit C .
2. Under $(\vec{b}_1, \vec{b}_2) : b_0 = 1 \Rightarrow C(\vec{b}_1) = 1$.

Thus $\langle C(\vec{x}) \rangle \in \text{BC-SAT}$.

LESSON LEARNED: We have to make sure that the reduction is poly-time computable!

SIMILAR PITFALL: $f_{L' \rightarrow L}$ cannot possibly know whether $x \in L'$ or $x \notin L'$ unless $L' \in \text{P}$! This is never the case if you are trying to prove that $L \in \text{NPC}$ (IMPORTANT).

Next reduction is a special case of a large family: reductions by restriction.

These reductions are used to prove that an NPC problem stays NPC even if we restrict the set of admissible instances.

DEF Given a set of boolean variables $\{x_1, x_2, \dots, x_n\}$:

- A literal is either a variable x_i or its negation $\neg x_j$ ($\bar{x}_j$). We will use y to denote a literal.
- A clause is a disjunction of 3 distinct literals, e.g. $C_i = x_1 \vee \bar{x}_3 \vee x_5$.
- A 3-CNF formula $\phi(x_1, \dots, x_n)$ is a conjunction of clauses built on the n variables.

E.g. $\phi(x_1, x_2, x_3, x_4) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$.

3-CNF-SAT:

I: $\langle \phi(x_1, x_2, \dots, x_n) = \bigwedge_{i=1}^m C_i \rangle$, ϕ 3-CNF formula.

Q: Is $\phi(\vec{x})$ satisfiable?

We will prove that $3\text{-CNF-SAT} \in \text{NPC}$.

Meaning: Even restricting the shape of the formula to a very simple one, the problem stays difficult!

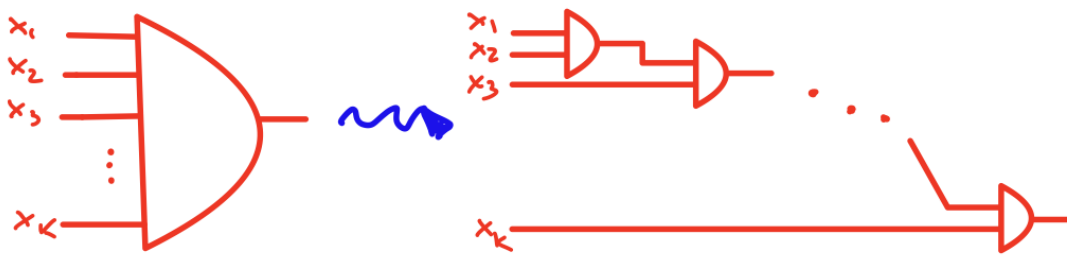
$3\text{-CNF-SAT} \in \text{NP}$ (trivial, exercise).

$3\text{-CNF-SAT} \in \text{NPH}$:

We reduce again from BC-SAT, using a similar idea to $\text{BC-SAT} \leq_p \text{SAT}$.

- **PREPROCESSING STEP:** we modify C to get an equivalent one:

Replace each AND or OR gate with fan-in ≥ 3 with a cascade of gates with fan-in 2, e.g.,



The resulting circuit C' is equivalent to C . Moreover $|\langle C' \rangle| = \Theta(|\langle C \rangle|)$ (# new gates $\leq$ # wires of C).

- Next, we write the formula $\psi(\vec{x}, \vec{y}) \wedge y_0$ as done for formula satisfiability. We have four types of subexpressions Γ :

$$y_i \leftrightarrow w_i^1 \vee w_i^2, \quad y_i \leftrightarrow w_i^1 \wedge w_i^2, \quad y_i \leftrightarrow \bar{w}_i, \quad y_0.$$

(w_i either x_k or y_j .)

- We can re-write each subexpression as a conjunction of disjunctions:

$$\begin{aligned}
y_i \leftrightarrow w_i^1 \vee w_i^2 &\equiv (y_i \vee \bar{w}_i^1) \wedge (y_i \vee \bar{w}_i^2) \wedge (\bar{y}_i \vee w_i^1 \vee w_i^2), \\
y_i \leftrightarrow w_i^1 \wedge w_i^2 &\equiv (\bar{y}_i \vee w_i^1) \wedge (\bar{y}_i \vee w_i^2) \wedge (y_i \vee \bar{w}_i^1 \vee \bar{w}_i^2), \\
y_i \leftrightarrow \bar{w}_i &\equiv (y_i \vee w_i) \wedge (\bar{y}_i \vee \bar{w}_i).
\end{aligned}$$

Equivalence verified via truth tables (HOMEWORK).

E.g.,

y_i	w_i	$y_i \leftrightarrow \bar{w}_i$	$A : y_i \vee w_i$	$B : \bar{y}_i \vee \bar{w}_i$	$A \wedge B$
0	0	0	0	1	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0

$$y_i \leftrightarrow \bar{w}_i = A \wedge B.$$

- After this transformation, $\psi(x; y) \wedge y_0$ has been rewritten as a conjunction of disjunctive clauses. However, some of the clauses have < 3 literals!
- We add “dummy” variables to bring all clauses to 3 literals:

$$\begin{aligned}
(y \vee w) &= (y \vee w) \vee 0 = (y \vee w) \vee (z \wedge \bar{z}) \\
&= (y \vee w \vee z) \wedge (y \vee w \vee \bar{z}).
\end{aligned}$$

(For clause y_0 repeat twice.)

- We have obtained a 3-CNF-FORMULA $\phi_{C'}(\vec{x}, \vec{y}, \vec{z})$ equivalent to $\psi_{C'}(\vec{x}, \vec{y}) \wedge y_0$.

$$\begin{aligned}
C \in \text{BC-SAT} &\iff C' \in \text{BC-SAT} \\
&\iff \psi_{C'}(\vec{x}, \vec{y}) \wedge y_0 \in \text{SAT} \\
&\iff \phi_{C'}(\vec{x}, \vec{y}, \vec{z}) \in \text{3-CNF-SAT}.
\end{aligned}$$

LESSON LEARNED: Reductions by restriction must “modify” f so that it maps into the set of restricted instances.

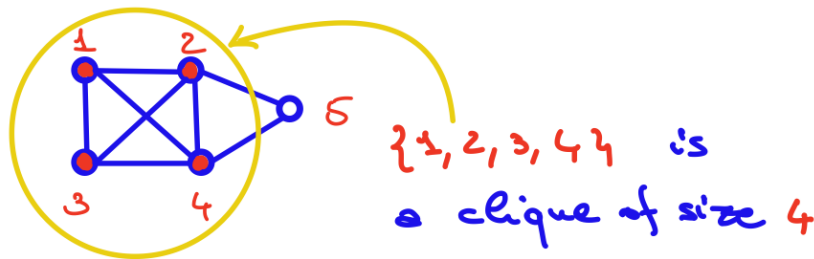
PROBLEMS ON GRAPHS

We will prove the NP-Completeness of three decision problems on graphs, related to very important optimization problems.

DEF Given $G = (V, E)$ undirected, a clique of size k is a subset $V' \subseteq V$ of k nodes such that each pair of distinct nodes in V' is connected by an edge (clique: complete subgraph).

$$V' \subseteq V : (|V'| = k) \wedge (\forall u \neq v \in V : (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \in E).$$

EXAMPLE:



We have:

CLIQUE

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain a clique of size k ?

The optimization version of this problem requires determining the clique of max size.

QUESTION: Why doesn't the decision problem ask "size $\geq k$ "? [Argue that the two questions are equivalent.]

Important application in bioinformatics, computational chemistry, social network analysis.

Let's prove that CLIQUE $\in$ NPC.

1. CLIQUE $\in$ NP trivial. Candidate certificate: set of k nodes. [Write $V_{\text{CLIQUE}}(x, y)$ as an exercise.]
2. CLIQUE $\in$ NPH. Reduction from 3-CNF-SAT.

$$\underbrace{\text{3-CNF-SAT}}_{\text{Known NPH}} \leq_p \underbrace{\text{CLIQUE}}_{\text{candidate}}.$$

We start from $\phi(x_1, x_2, \dots, x_m) = C_1 \wedge C_2 \wedge \dots \wedge C_m$ with $C_i = y_1^i \vee y_2^i \vee y_3^i$, $y_j^i \in \{x_k, \bar{x}_k : 1 \leq k \leq n\}$.

We have to define a reduction function that takes ϕ and returns an instance of CLIQUE in poly-time:

$$f(\langle \phi(x_1, \dots, x_n) \rangle) = \langle G_\phi, k_\phi \rangle.$$

1. We set $k_\phi = m$ (# of clauses of ϕ).
2. $G_\phi = (V_\phi, E_\phi)$.

V_ϕ : (One node for each literal y_j^i , $1 \leq i \leq m$, $1 \leq j \leq 3$):

$$V_\phi = \{v_j^i : 1 \leq i \leq m, 1 \leq j \leq 3\} \Rightarrow |V_\phi| = 3m.$$

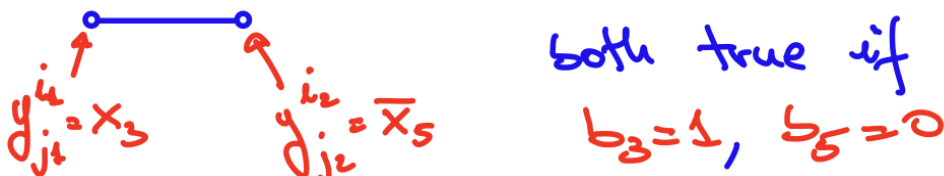
E_ϕ :

$$\{v_{j_1}^{i_1}, v_{j_2}^{i_2}\} \in E_\phi \iff (i_1 \neq i_2) \wedge (y_{j_1}^{i_1} \neq \neg y_{j_2}^{i_2}).$$

There is an edge between two nodes if their corresponding literals belong to distinct clauses and are not the negation of each other (e.g. $y_{j_1}^{i_1} = x_1$, $y_{j_2}^{i_2} = \bar{x}_1$).

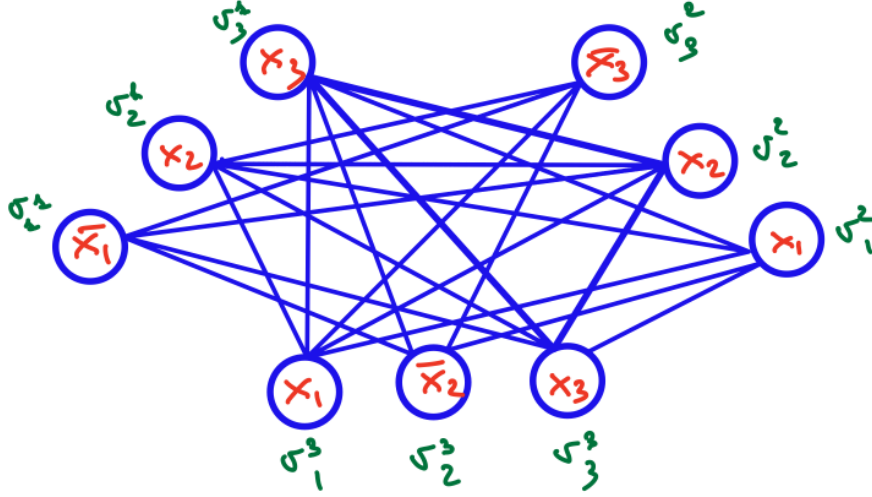
INTUITION: Two literals corresponding to an edge can be made both true under the same truth assignment.

E.g.,



EXAMPLE:

$$\phi(x_1, x_2, x_3) = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3).$$



Running time: $\Theta(m^2) = \Theta(|\langle \phi \rangle|^2)$.

Each node is compared to all other nodes to determine the edges.

$\Rightarrow f$ is ptc.

RECAP

- BC-SAT $\in$ NPC (Cook, 1971).
- SAT $\in$ NPC (BC-SAT $\leq_p$ SAT, make sure that $f(x)$ is polynomial).
- 3-CNF-SAT $\in$ NPC (reduction by restriction).
- CLIQUE $\in$ NPC (3-CNF-SAT $\leq_p$ CLIQUE, “context switch”):

$$f(\langle \phi(x_1, \dots, x_n) = \bigwedge_{i=1}^m C_i \rangle) = \langle G_\phi, k_\phi \rangle, \quad C_i = y_1^i \vee y_2^i \vee y_3^i.$$

1. $k_\phi = m$.

2. $G_\phi = (V_\phi, E_\phi)$.

$$V_\phi = \{v_j^i : 1 \leq i \leq m, 1 \leq j \leq 3\} \Rightarrow |V_\phi| = 3m.$$

One node per literal occurrence.

E_ϕ :

$$\{v_{j_1}^{i_1}, v_{j_2}^{i_2}\} \in E_\phi \iff (i_1 \neq i_2) \wedge (y_{j_1}^{i_1} \neq \neg y_{j_2}^{i_2}).$$

Edges between literals of different clauses that can both be True under the same truth assignment.

We have to show that:

$$\langle \phi \rangle \in \text{3-CNF-SAT} \iff f(\langle \phi \rangle) = \langle G_\phi, k_\phi \rangle \in \text{CLIQUE}.$$

($\Rightarrow$) $\langle \phi \rangle \in \text{3-CNF-SAT}$: there exists a satisfying assignment $\vec{b} \in \{0, 1\}^n : \phi(\vec{b}) = 1$. Since ϕ is in 3-CNF, all clauses must be True under $\vec{b} \Rightarrow$ under $\vec{b}$ there is a true literal in each clause. Let these literals be:

$$\{y_{j_1}^1, y_{j_2}^2, \dots, y_{j_m}^m\}, \quad 1 \leq j_i \leq 3, \quad 1 \leq i \leq m,$$

and let $V' = \{v_{j_1}^1, v_{j_2}^2, \dots, v_{j_m}^m\}$ (corresponding nodes in G_ϕ).

For two distinct nodes $v_{j_{i_1}}^{i_1}, v_{j_{i_2}}^{i_2} \in V'$, we have:

$$(i_1 \neq i_2) \wedge (y_{j_{i_1}}^{i_1} \text{ and } y_{j_{i_2}}^{i_2} \text{ both True under } \vec{b}) \Rightarrow \{v_{j_{i_1}}^{i_1}, v_{j_{i_2}}^{i_2}\} \in E.$$

Thus V' is a CLIQUE of size $k_\phi = m$.

$$\Rightarrow f(\langle \phi \rangle) = \langle G_\phi, k_\phi \rangle \in \text{CLIQUE}.$$

$$(\Leftarrow) f(\langle \phi \rangle) = \langle G_\phi, k_\phi \rangle \in \text{CLIQUE}.$$

Let us build a satisfying assignment $\vec{b}$ starting from a clique $V' = \{v_{j_1}^1, v_{j_2}^2, \dots, v_{j_m}^m\}$.

For $1 \leq i \leq m$:

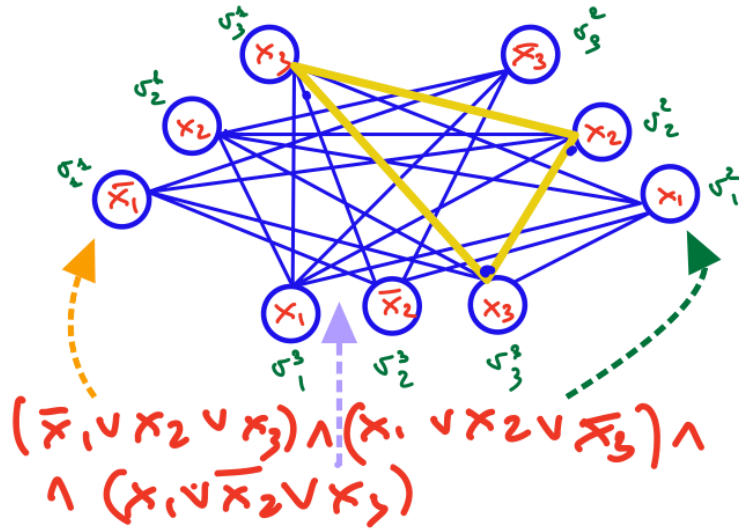
1. If $y_{j_i}^i = x_k$ then set $b_k = 1$.
2. If $y_{j_i}^i = \bar{x}_k$ then set $b_k = 0$.

If some value b_ℓ is still unfixed, set it to an arbitrary value.

Assignment $\vec{b}$ is well-defined since I cannot set $b_k = 0$ for some i_1 and $b_k = 1$ for some i_2 , or otherwise $y_{j_{i_1}}^{i_1} = \bar{y}_{j_{i_2}}^{i_2}$ (impossible, since V' is a clique).

Under $\vec{b}$ there is at least one true literal in each clause

$$\Rightarrow \phi(\vec{b}) = 1 \Rightarrow \langle \phi \rangle \in \text{3-CNF-SAT}.$$



A 3-clique contains 3 literals (one per clause) that can be made all True under the same truth assignment.

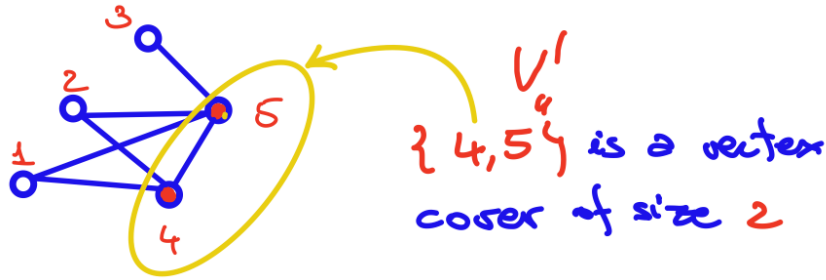
$$v_3^1 \leftrightarrow x_3 \longrightarrow b_3 = 1, \quad v_2^2 \leftrightarrow x_2 \longrightarrow b_2 = 1, \quad v_3^3 \leftrightarrow x_3 \longrightarrow b_3 = 1, \quad b_1 = 0/1.$$

We introduce two extra problems on graphs:

DEF: Given $G = (V, E)$ undirected, a vertex cover of size k is a subset of nodes $V' \subseteq V$, $|V'| = k$ such that each edge in E has at least one endpoint in V' .

$$V' \subseteq V, |V'| = k : \forall u \neq v \in V : \\ \{u, v\} \in E \Rightarrow (u \in V') \vee (v \in V').$$

EXAMPLE:



We have:

VERTEX COVER (VC)

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain a vertex cover of size k ?

The optimization version of this problem requires determining the vertex cover of min size.

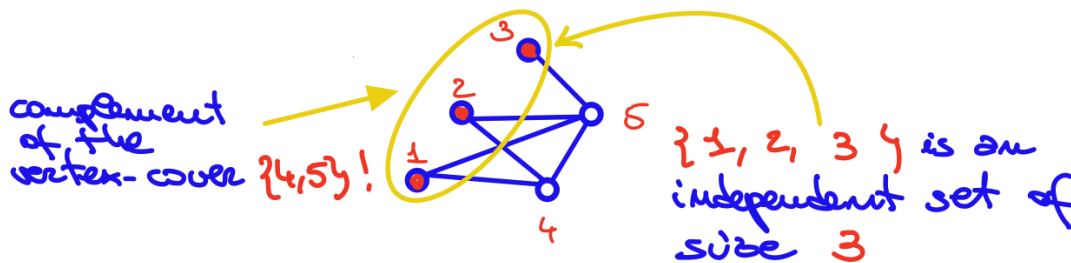
A vertex cover identifies a (possibly small) set of nodes from where all edges can be controlled. Crucial in networking, distributed computing, etc...

Second problem:

DEF: Given $G = (V, E)$ undirected, an independent set of size k is a subset of nodes $V' \subseteq V$, $|V'| = k$ such that there is no edge in E with both endpoints in V' !

$$V' \subseteq V, |V'| = k : \forall u \neq v \in V : \\ (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \notin E.$$

EXAMPLE:



We have:

INDEPENDENT SET (IS)

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain an independent set of size k ?

The optimization version of this problem requires determining the independent set of max size.

An independent set is somewhat the “negative” of a clique, with no edges connecting any two of its nodes. Important primitive for graph-based computations.

The three problems share many similarities. We will see that reductions will be mostly “syntactic”, based on logical manipulation of the definitions of clique, vertex-cover, independent-set.

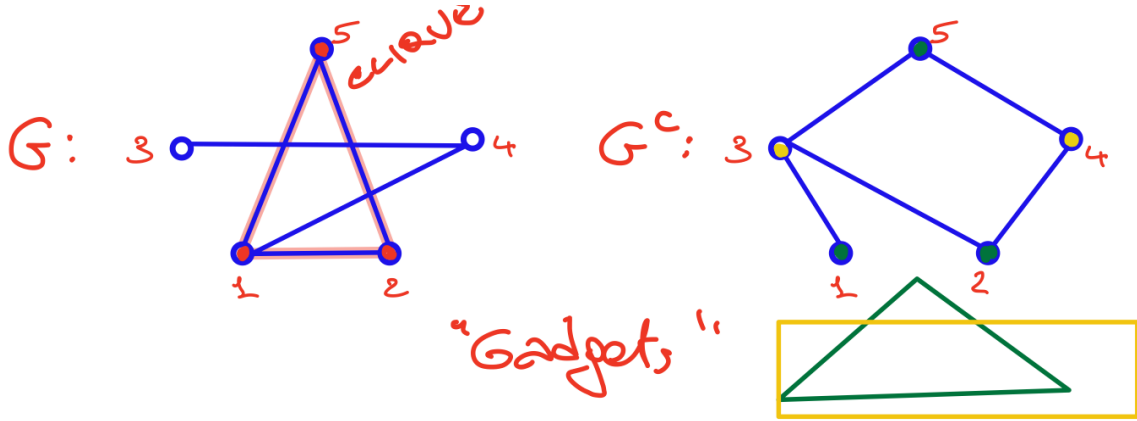
We will make use of the complement of a graph.

DEF: Given $G = (V, E)$ its complement $G^c = (V, E^c)$ is the graph with the same node-set and

$$E^c : \{ \{u, v\} : u \neq v \in V, \{u, v\} \notin E \}.$$

G^c is the “negative” of G w.r.t. edges.

EXAMPLE



OBSERVE: Clique $\{1, 2, 5\}$ in G implies that:

1. $V - \{1, 2, 5\} = \{3, 4\}$ is a vertex cover in G^c (all edges between $\{1, 2, 5\}$ disappear in E^c , thus each edge in E^c has an endpoint in $\{3, 4\}$).
2. $\{1, 2, 5\}$ is an independent set in G^c (there cannot be edges between $\{1, 2, 5\}$ in E^c).

These observations suggest the following two reductions!

1. $\text{CLIQUE} \leq_p \text{VC}$ using:

$$f_{C \rightarrow \text{VC}}(\langle G = (V, E), k \rangle) = \langle G^c = (V, E^c), |V| - k \rangle.$$

2. $\text{CLIQUE} \leq_p \text{IS}$ using:

$$f_{C \rightarrow \text{IS}}(\langle G = (V, E), k \rangle) = \langle G^c = (V, E^c), k \rangle.$$

Clearly, $f_{C \rightarrow \text{VC}}$ and $f_{C \rightarrow \text{IS}}$ are ptc (the construction of $\langle G^c, |V| - k \rangle$ and $\langle G^c, k \rangle$ requires time $O(|\langle G, k \rangle|^2)$).

Let's prove that $f_{C \rightarrow \text{VC}}$ is a reduction by logical manipulations of the definition of CLIQUE:

$$x = \langle G = (V, E), k \rangle \in \text{CLIQUE}$$

$$\iff \exists V' \subseteq V, (|V'| = k) \wedge (\forall u \neq v \in V : (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \in E).$$

{transform using contrapositive} + $\{\neg(a \wedge b) = \neg a \vee \neg b\}$

$$\iff \exists V' \subseteq V, (|V'| = k) \wedge (\forall u \neq v \in V' : \{u, v\} \notin E \Rightarrow (u \notin V') \vee (v \notin V')).$$

$$\{u, v\} \notin E \iff \{u, v\} \in E^c, \quad u \notin V' \iff u \in V - V'.$$

$$\iff \exists V' \subseteq V, (|V'| = k) \wedge (\forall u \neq v \in V' : \{u, v\} \in E^c \Rightarrow (u \in V - V') \vee (v \in V - V')).$$

{now, set $V'' = V - V' : |V''| = |V| - k$ }

$$\iff \exists V'' \subseteq V, (|V''| = |V| - k) \wedge (\forall u \neq v \in V' : \{u, v\} \in E^c \Rightarrow (u \in V'') \vee (v \in V'')).$$

{this says that V'' is a VC of G^c }

$$\iff f(x) = \langle G^c = (V, E^c), |V| - k \rangle \in \text{VC}.$$

REMARK: The proof proceeds by logical equivalences (can be read both $\Downarrow$ and $\Uparrow$).

Let's use the same technique to prove that $f_{C \rightarrow \text{IS}}$ is a reduction:

$$\begin{aligned} x &= \langle G = (V, E), k \rangle \in \text{CLIQUE} \\ &\iff \exists V' \subseteq V, (|V'| = k) \wedge (\forall u \neq v \in V' : (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \in E) \\ &\iff \exists V' \subseteq V, (|V'| = k) \wedge (\forall u \neq v \in V' : (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \notin E^c). \end{aligned}$$

{this says that V' is an IS of G^c }

$$\iff f(x) = \langle G^c = (V, E^c), k \rangle \in \text{IS}.$$

EXERCISE Using the same technique, prove that $\text{VC} \leq_p \text{IS}$.

These reductions suggest that CLIQUE, VC and IS are mostly equivalent problems BUT this is true only for what concerns exact solutions! We will argue that the optimization versions of the three problems are radically different w.r.t. approximation. In particular:

MINIMUM VC is “easy” to approximate.

MAXIMUM CLIQUE/IS are “hard” to approximate.

We now introduce a problem in the realm of COMPUTER ARITHMETIC:

SUBSET-SUM (SS)

I: $\langle S, t \rangle : S \subseteq \mathbb{N}$ finite, $t \in \mathbb{N}$ (“target”).

Q: $\exists S' \subseteq S : \sum_{s \in S'} s = t$?

The problem asks whether there is a subset of the input set whose sum is equal to t (the target).

EXAMPLE:

$$\langle \{10, 15, 4, 3, 17\}, 24 \rangle \in L_{\text{SS}}$$

since $24 = 4 + 3 + 17$ ($S' = \{3, 4, 17\}$).

REMARK: $|\langle S, t \rangle|$ is the total number of bits needed to encode $s \in S \forall s$ and t :

$$|\langle S, t \rangle| = \Theta\left(\sum_{s \in S} \log s + \log t\right).$$

Clearly, $\text{SS} \in \text{NP}$.

Candidate certificate: $y = \langle S' \rangle$, $S' \subseteq S$.

Let us prove that $\text{SS} \in \text{NPH}$ by proving that $3\text{-CNF-SAT} \leq_p \text{SS}$.

The reduction is much more complex than the other ones seen before.

Given a 3-CNF formula:

$$\phi(x_1, x_2, \dots, x_n) = C_1 \wedge C_2 \wedge \dots \wedge C_m,$$

with $C_i = y_1^i \vee y_2^i \vee y_3^i$, $y_j^i \in \{x_k, \bar{x}_k : 1 \leq k \leq n\}$, the reduction f must create $\langle S_\phi, t_\phi \rangle$ such that

$$\langle \phi \rangle \in 3\text{-CNF-SAT} \iff \langle S_\phi, t_\phi \rangle \in \text{SS}.$$

f creates a set S_ϕ of $2n + 2m$ numbers, each with $n + m$ decimal digits:

$$S_\phi = \underbrace{\left\{ \boxed{n+m} \quad \boxed{n+m} \quad \boxed{n+m} \quad \dots \quad \boxed{n+m} \right\}}_{2n+2m}.$$

The n most significant digits correspond to the n variables $x_1, \dots, x_n$. The m least significant digits correspond to the m clauses.

CRUCIAL PROPERTY: the sum of all digits in each position i , $1 \leq i \leq n + m$, does never generate a carry.

This is important to relate the satisfiability of ϕ to the achievement of the target t_ϕ .

With no loss of generality:

1. Assume that each variable x_k of $\phi(x_1, \dots, x_n)$ appears in at least one clause (as $y_j^i = x_k$ or $y_j^i = \bar{x}_k$).
2. Assume that no clause contains x_k and $\bar{x}_k$.

Otherwise: Eliminate variable (1) or clause (2) ($x_k \vee \bar{x}_k = 1!$).

We create:

- two numbers per variable x_i : δ_i, δ'_i , $1 \leq i \leq n$ ($2n$).
- two numbers per clause C_j : s_j, s'_j , $1 \leq j \leq m$ ($2m$).

$\forall 1 \leq i \leq n$:

- δ_i, δ'_i have a single 1 in the i th position of the n most significant digits.
- δ_i has 1's in all positions j of the m least significant digits, where C_j contains literal x_i .
- δ'_i has 1's in all positions j of the m least significant digits, where C_j contains literal $\bar{x}_i$.
- All other digits are 0.

$\forall 1 \leq j \leq m$:

- s_j has a single 1 in the j th position of the m least significant ones.
- s'_j has a single 2 in the j th position of the m least significant ones.
- All other digits are 0.

Finally, $t_\phi = \underbrace{1 \cdots 1}_n \underbrace{44 \cdots 4}_m$.

$n = 2, m = 2$: 1144.

(Recall that all numbers have to be interpreted to the base 10.)

EXAMPLE:

$$\phi(x_1, x_2, x_3) = \underbrace{(x_1 \vee \bar{x}_2 \vee x_3)}_{C_1} \wedge \underbrace{(x_1 \vee x_2 \vee x_3)}_{C_2}.$$

S_ϕ :

	x_1	x_2	x_3	C_1	C_2	
δ_1	1	0	0	1	1	10.011
δ'_1	1	0	0	0	0	10.000
δ_2	0	1	0	0	1	1.001
δ'_2	0	1	0	1	0	1.010
δ_3	0	0	1	1	1	111
δ'_3	0	0	1	0	0	100
s_1	0	0	0	1	0	10
s'_1	0	0	0	2	0	20
s_2	0	0	0	0	1	1
s'_2	0	0	0	0	2	2
t_ϕ	1	1	1	4	4	11.144

There are exactly 3 1's in each clause column (the δ_i, δ'_i rows).

The sum in each clause column is 3 (the s_j, s'_j rows).

Observe that the sum of all numbers yields

$$\underbrace{22 \cdots 2}_n \underbrace{66 \cdots 6}_m.$$

Two 1's due to δ_i and δ'_i in the i th column.

3 1's due to literals plus 1 and 2 due to s_j & s'_j in the j th column.

No carry can ever be generated by summing any subset of numbers $\Rightarrow$ no “interference” between different columns.

The running time of the reduction is proportional to the number of digits generated:

$$(\underbrace{2n}_{\delta_i, \delta'_i} + \underbrace{2m}_{s_j, s'_j} + \underbrace{1}_{t_\phi}) \cdot (n + m) = O((n + m)^2) = O(|\langle \phi \rangle|^2).$$

NOTE: After we generate the numbers in decimal, we can encode them in binary without changing the order of magnitude of the size of the encoding.

Let us prove that

$$\langle \phi(x_1, \dots, x_n) = \bigwedge_{j=1}^m C_j \rangle \in \text{3-CNF-SAT} \iff f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in \text{SS}.$$

($\Rightarrow$) $\phi \in \text{3-CNF-SAT} \Rightarrow \exists \vec{b} : \phi(\vec{b}) = 1$. Let us build a subset $S'_\phi \subseteq S_\phi : \sum_{s \in S'_\phi} s = t_\phi$.

For each $1 \leq i \leq n$:

- if $b_i = 1$ then $\delta_i \in S'_\phi$;
- if $b_i = 0$ then $\delta'_i \in S'_\phi$.

After this selection of n values:

1. The sum of the values in S'_ϕ will have 1's in the n most significant digits (exactly one between δ_i and δ'_i is selected for $1 \leq i \leq n$).
2. The digit of the sum in each clause column j (m least significant digits) reflects the number of true literals in C_j under $\vec{b}$ (δ_i has 1's for clauses containing x_i , while δ'_i has 1's for clauses containing $\bar{x}_i$).

Observe that since $\phi(\vec{b}) = 1$, thanks to 2), each sum digit in clause positions is > 0 ! (either 1, 2, or 3).

Then $\forall j, 1 \leq j \leq m$, I can “adjust” the digit in the j th position by adding to S'_ϕ :

1. s_j , if the sum digit is 3 (3 true literals);
2. s'_j , if the sum digit is 2 (2);
3. s_j AND s'_j , if the sum digit is 1 (1).

Now, in $\sum_{s \in S'_\phi} s$, the digit in each of the m least significant positions is 4!

Thus,

$$\sum_{s \in S'_\phi} s = \underbrace{1 \cdots 1}_n \underbrace{4 \cdots 4}_m = t_\phi,$$

therefore $f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in \text{SS}$.

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $\vec{b} = (1, 0, 1) \Rightarrow \phi(\vec{b}) = 1$.

We put $\delta_1, \delta'_2, \delta_3$ in S'_ϕ :

$$\{\delta_1 = 10011, \delta'_2 = 01010, \delta_3 = 00111\}.$$

The sum in the column relative to

C_1 is 3 $\Rightarrow$ we add $s_1 = 00010$.

C_2 is 2 $\Rightarrow$ we add $s'_2 = 00002$.

$$\sum_{s \in S'_\phi} s = \begin{array}{r} 10011 \quad + \quad \delta_1 + \\ 1010 \quad + \quad \delta'_2 + \\ 111 \quad + \quad \delta_3 + \\ 10 \quad + \quad s_1 + \\ 2 \quad + \quad s'_2 + \\ \hline 11144 \quad = t_\phi \end{array}$$

($\Leftarrow$) Let $f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in \text{SS}$ then $\exists S'_\phi : \sum_{s \in S'_\phi} s = t_\phi$.

1. Since the sum digit in each of the n most significant digits is 1, there is either δ_i or δ'_i in S'_ϕ , for $1 \leq i \leq n$. Let

$$b_i = \begin{cases} 1 & \text{if } \delta_i \in S'_\phi, \\ 0 & \text{if } \delta'_i \in S'_\phi. \end{cases}$$

In each clause column j , since the sum digit is 4, there is a contribution ≥ 1 coming from δ_i or δ'_i , for all $1 \leq i \leq n$ (or otherwise the sum digit would be $\leq 1 + 2 = 3!$).

$\Rightarrow$ under $\vec{b}$ there is at least one true literal for each clause

$$\Rightarrow \phi(\vec{b}) = 1 \Rightarrow \langle \phi \rangle \in \text{3-CNF-SAT}.$$

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $S'_\phi = \{\delta_1, \delta'_2, \delta_3, s_1, s'_2\}$. Then

$$\begin{array}{r} 10011 \quad + \quad \delta_1 + \\ 01010 \quad + \quad \delta'_2 + \\ 00111 \quad + \quad \delta_3 + \\ 00010 \quad + \quad s_1 + \\ 00002 \quad = \quad s'_2 + \\ \hline 11144 \quad = t_\phi \end{array}$$

We built $\vec{b} = (1, 0, 1)$. Under $\vec{b}$ there are 3 true literals in C_1 (only $s_1 \in S'_\phi$) and 2 true literals in C_2 (only $s'_2 \in S'_\phi$)